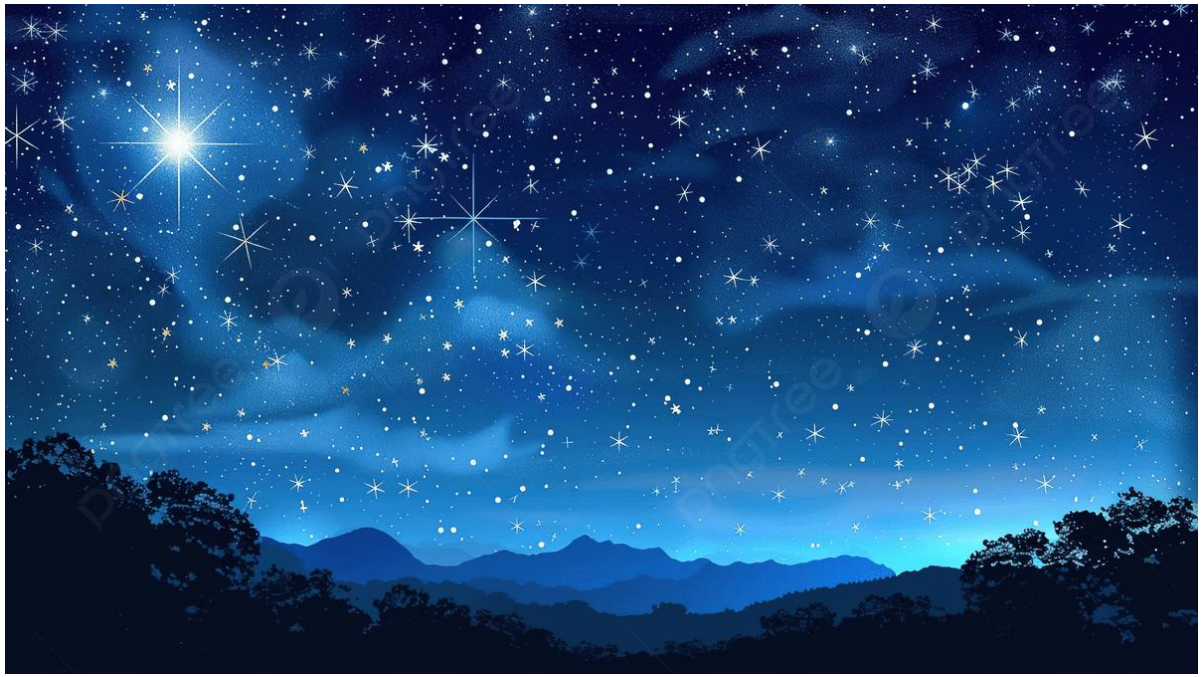
Blob Detection and SIFT

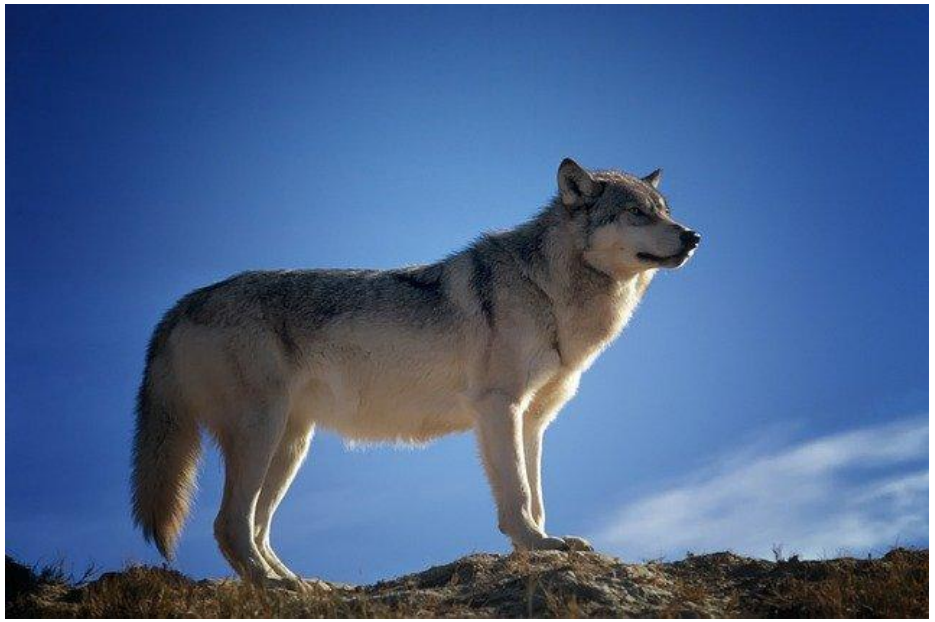


https://liaoux.github.io/uir-cv-2026/

Introduction to Computer Vision
Spring 2026, Lecture 5

A Little Quiz

in this image ?



Can you find the wolf's head

Template Matching: A First Solution

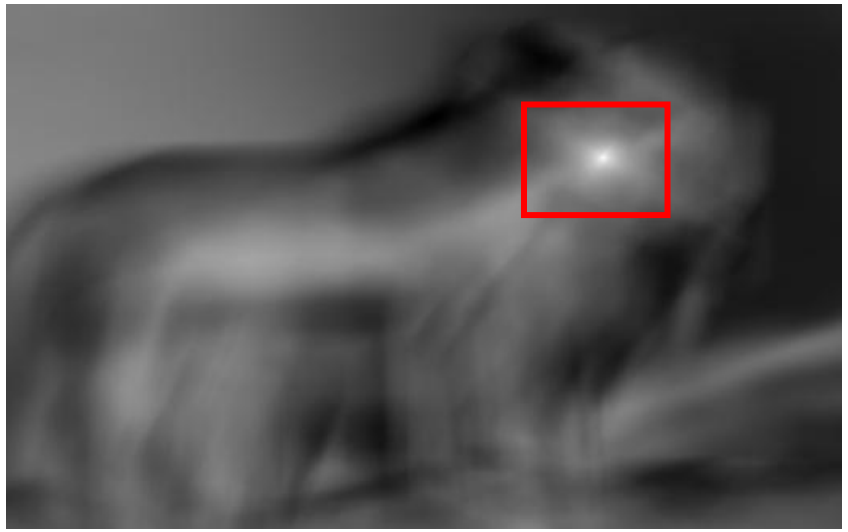
You can recognize the template in the image using Normalized Cross Correlation

image (f)

$\otimes$

template (t)

$=$

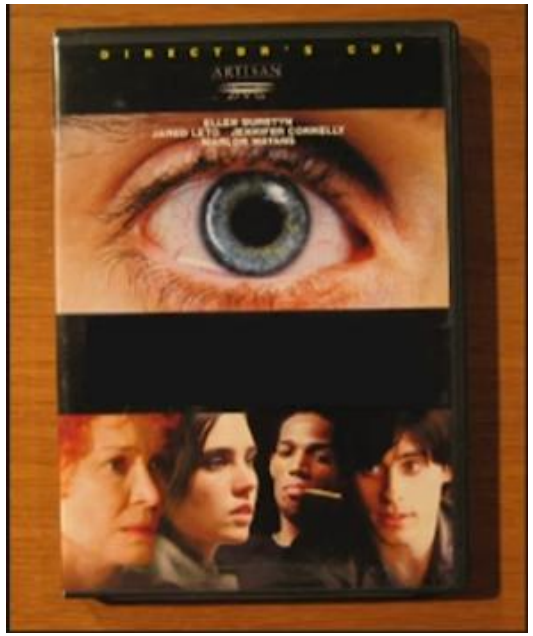


The brighter the pixel, the higher the correlation value. (the wolf's face)

Works well when the object appears at the **same scale and orientation**

Now a Harder Quiz...

Can you find this DVD cover in the cluttered scene?



Template

Rich 2D image

The object is:

- **rotated**
- **scaled differently**
- **partially occluded**

Would Template Matching still work ?

Why Template Matching Falls Short

- **Rotation** — the object appears at a different angle
- **Scale** — the object is larger or smaller than the template
- **Occlusion** — the object is partially hidden
- **Workaround**: create **many templates** for each rotation, scale, and partial view
  - → Computationally **impractical**, especially with many objects

The Solution: Find Distinctive Features

- Instead of matching the **whole object** → find and match **distinctive local features**
- If enough features match in the **same geometric configuration** → object is detected
- We can also recover **rotation and scale** from the matched features
- ➞ This is exactly what **SIFT** enables us to do

Find and Match "Interesting Points or Features"

SIFT: What is an Interesting Point?

- A question with a **long history** in computer vision *(Moravec, late 1970s)*
- Edges and corners → **not descriptive enough**
- Better notion: **"Blobs"**
  - A patch with a **well-defined local appearance**
  - Detected using **image derivatives**
  - Detectable across **multiple scales**

We will develop a theory of blob detection to build our way to SIFT

SIFT: What We Will Cover

In this lecture, we will cover:

1. **Blob Detection Theory** — the mathematical foundation of SIFT
2. **SIFT Detector** — how interest points are located in practice *(David Lowe)*
3. **SIFT Descriptor** — how local appearance is captured and described
   - Rotation and scale invariant
   - Robust to illumination changes
4. **Applications**
   - Image stitching
   - Object recognition

What is an Interest Point?

Raw Images are Hard to Match

patch 1

patch 2

Different size, orientation, lighting, brightness, etc.

Removing Sources of Variation

Source of variation: scale differences

patch 1

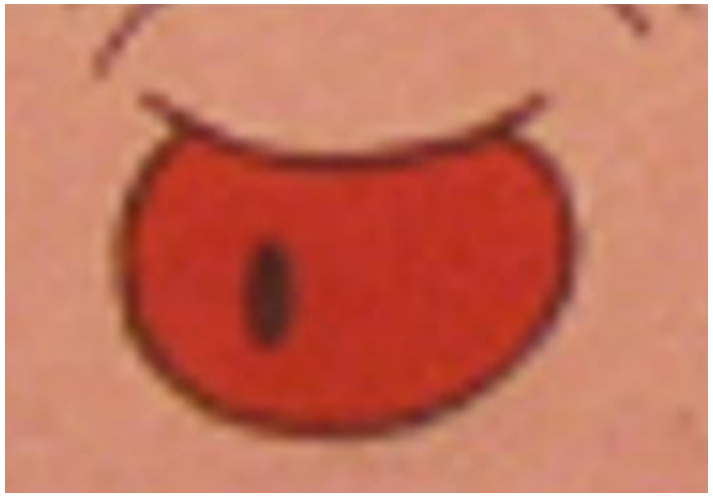
patch 2

re scaled patch 2

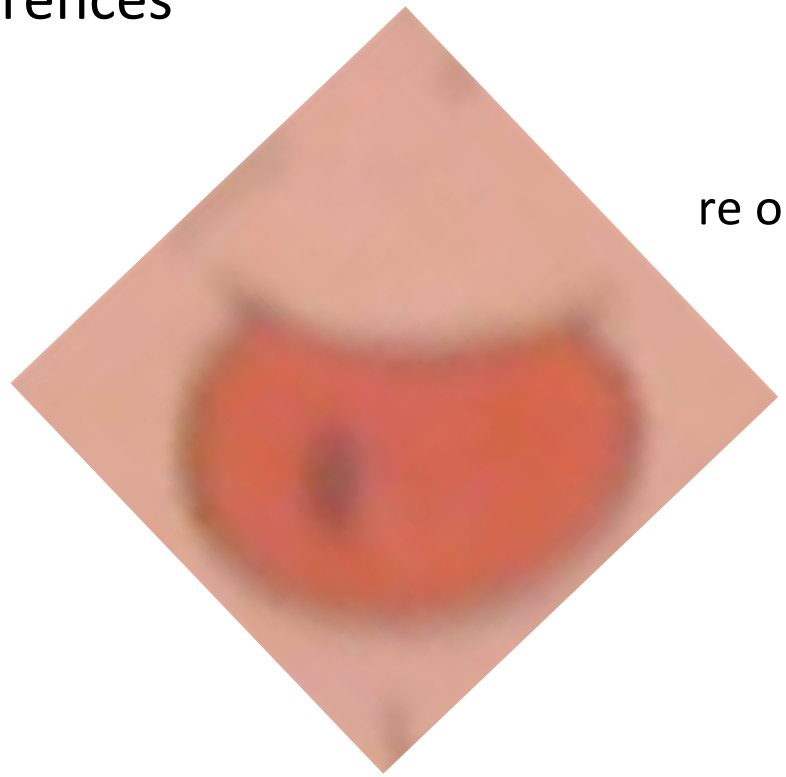
re scale patch 2 so it matches the scale of patch 1

Removing Sources of Variation

Source of variation: orientation differences



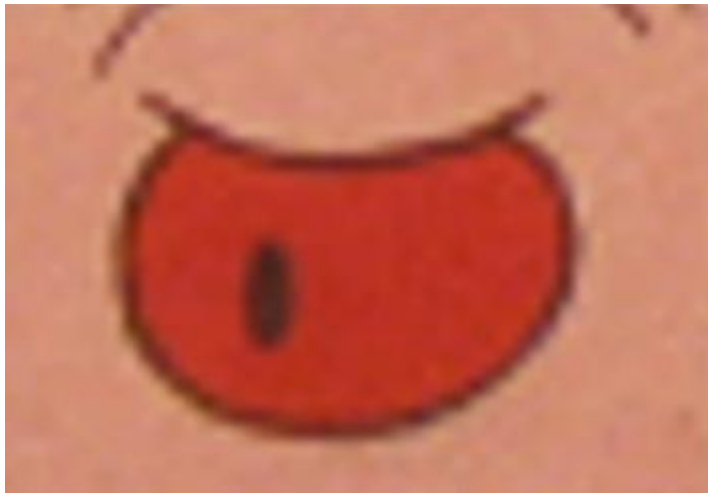
patch 1



re oriented patch 2

rotate the re scaled patch 2 so it matches the orientation of patch 1

Removing Sources of Variation

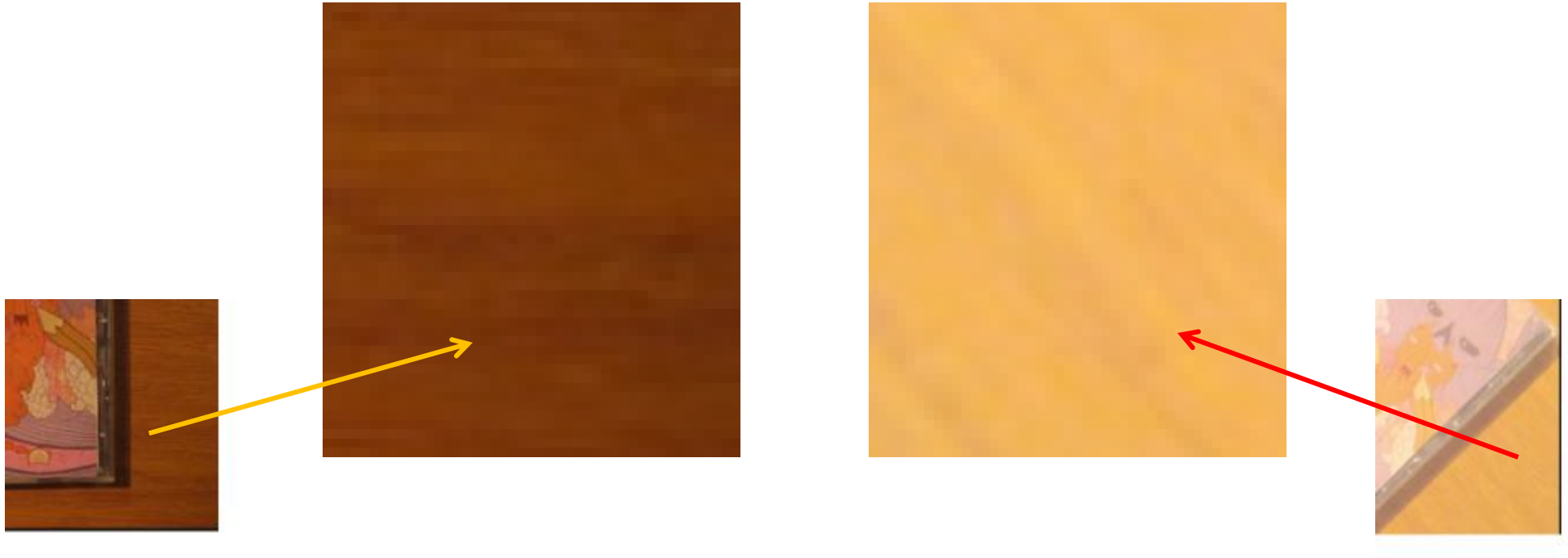


patch 1

matched patch 2

Matching becomes easier if we can remove variations like size, orientation and lighting

Some Patches are not “Interesting”



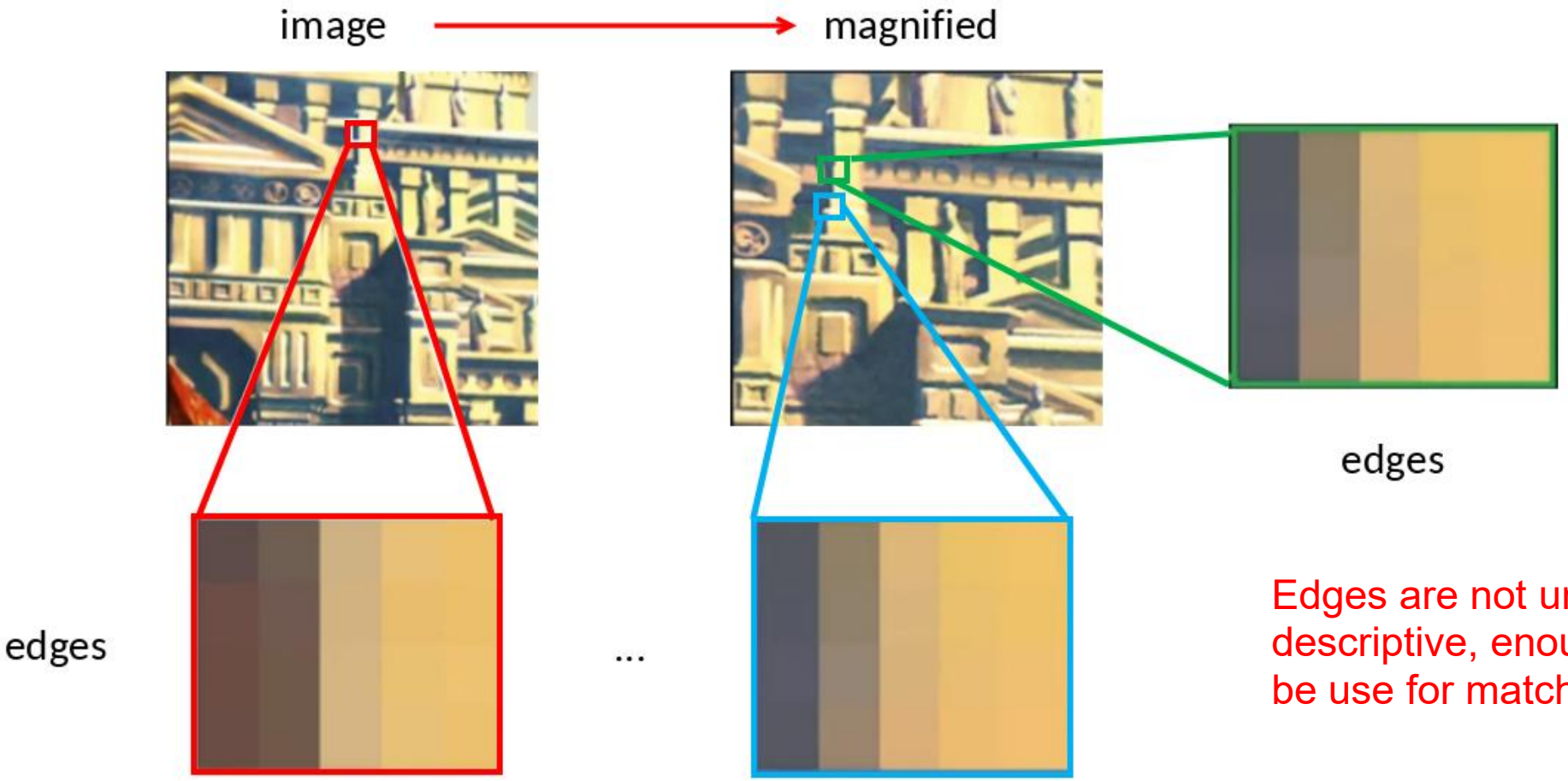
These patches include weak textures but they are neither unique nor interesting enough to use for matching.

What is an Interest Point/Feature?

An interest point must have the following attributes:

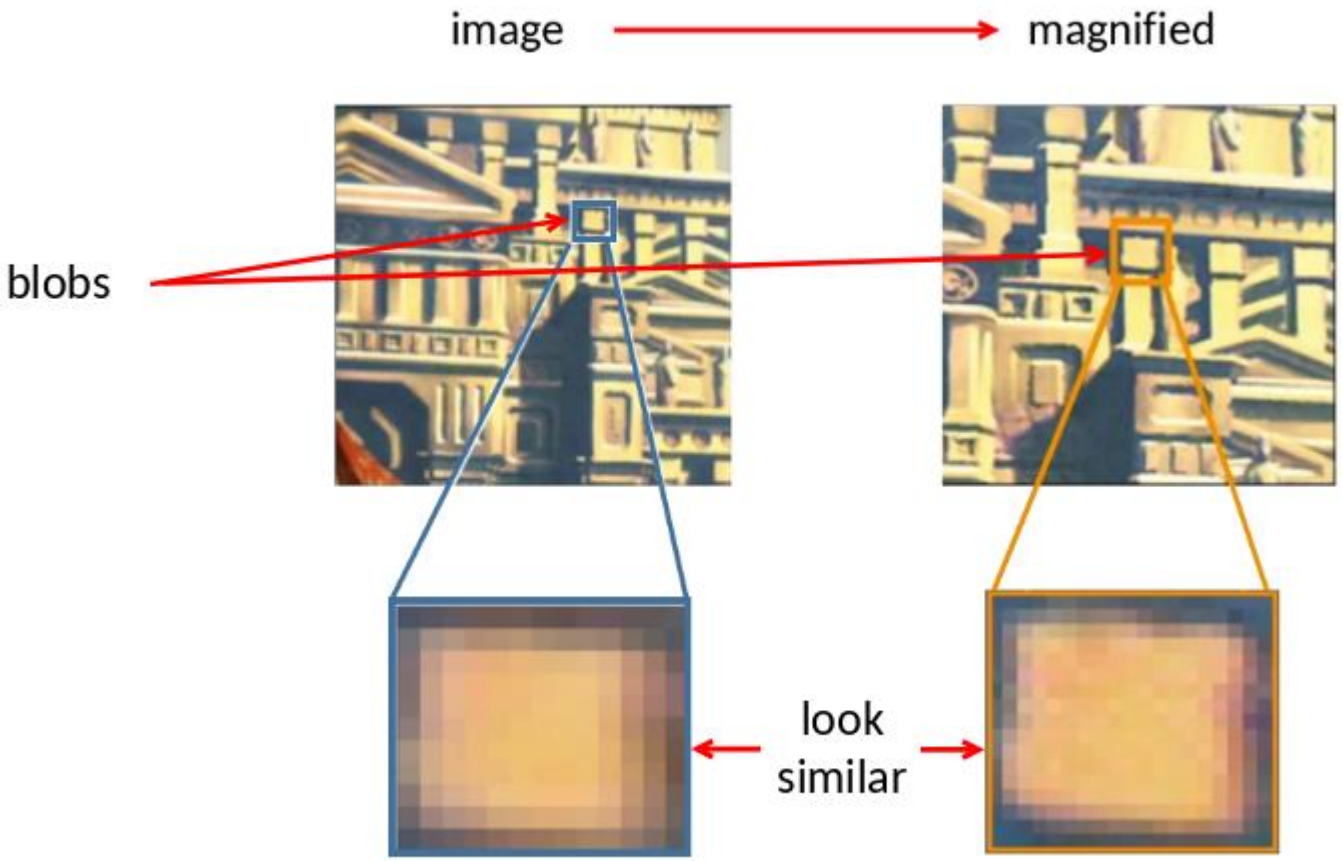
- Has rich image content (brightness variation, color variation, etc.) within the local window
- Has well-defined representation (signature) for matching/comparing with other points
- Has a well-defined position in the image
- Should be invariant to image rotation and scaling
- Should be insensitive to lighting changes

Are Lines/Edges Interesting?



Edges are not unique, or descriptive, enough to be use for matching.

Are Blobs Interesting?



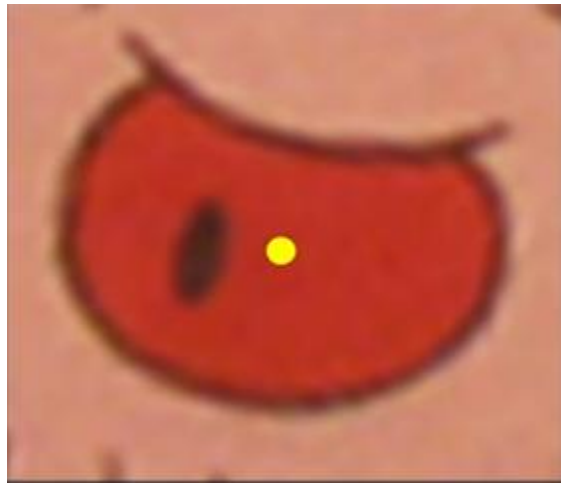
blobs are useful as they have some kind of a local appearance due to the brightness variation within them

Yes! Blobs have **fixed position** and definite **size**.

Blobs as Interest Points

For a Blob-like Feature to be useful, we need to:

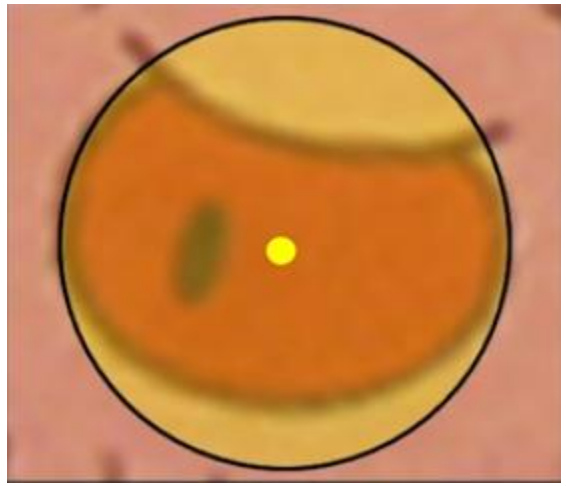
- **Locate** the blob:
  - A blob must have a well-defined position in the image, which emerges naturally from the blob detection process itself rather than from any specific feature inside it.



Blobs as Interest Points

For a Blob-like Feature to be useful, we need to:

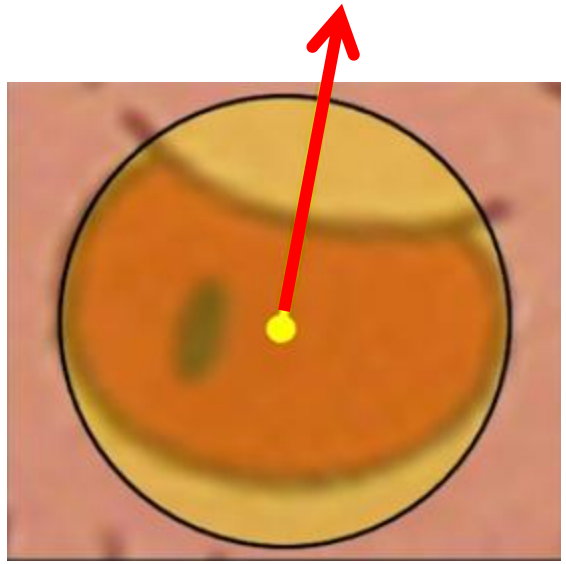
- **Locate** the blob
- Determine its **size:**
  - A blob must have a size associated with it, not related to its internal edges or boundaries, but rather representing the rough scale of its overall appearance.



Blobs as Interest Points

For a Blob-like Feature to be useful, we need to:

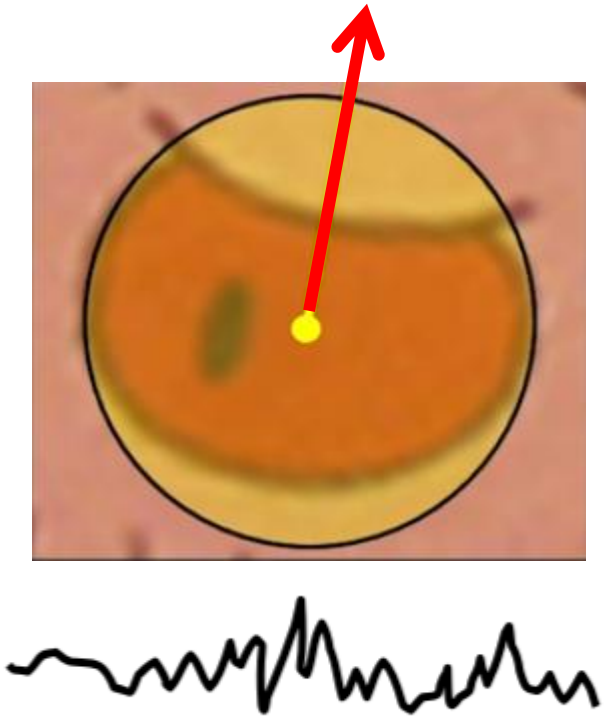
- **Locate** the blob
- Determine its **size**
- Determine its **orientation:**
  - A blob must have a computable principal orientation, allowing us to handle the same blob appearing at different angles in different images.



Blobs as Interest Points

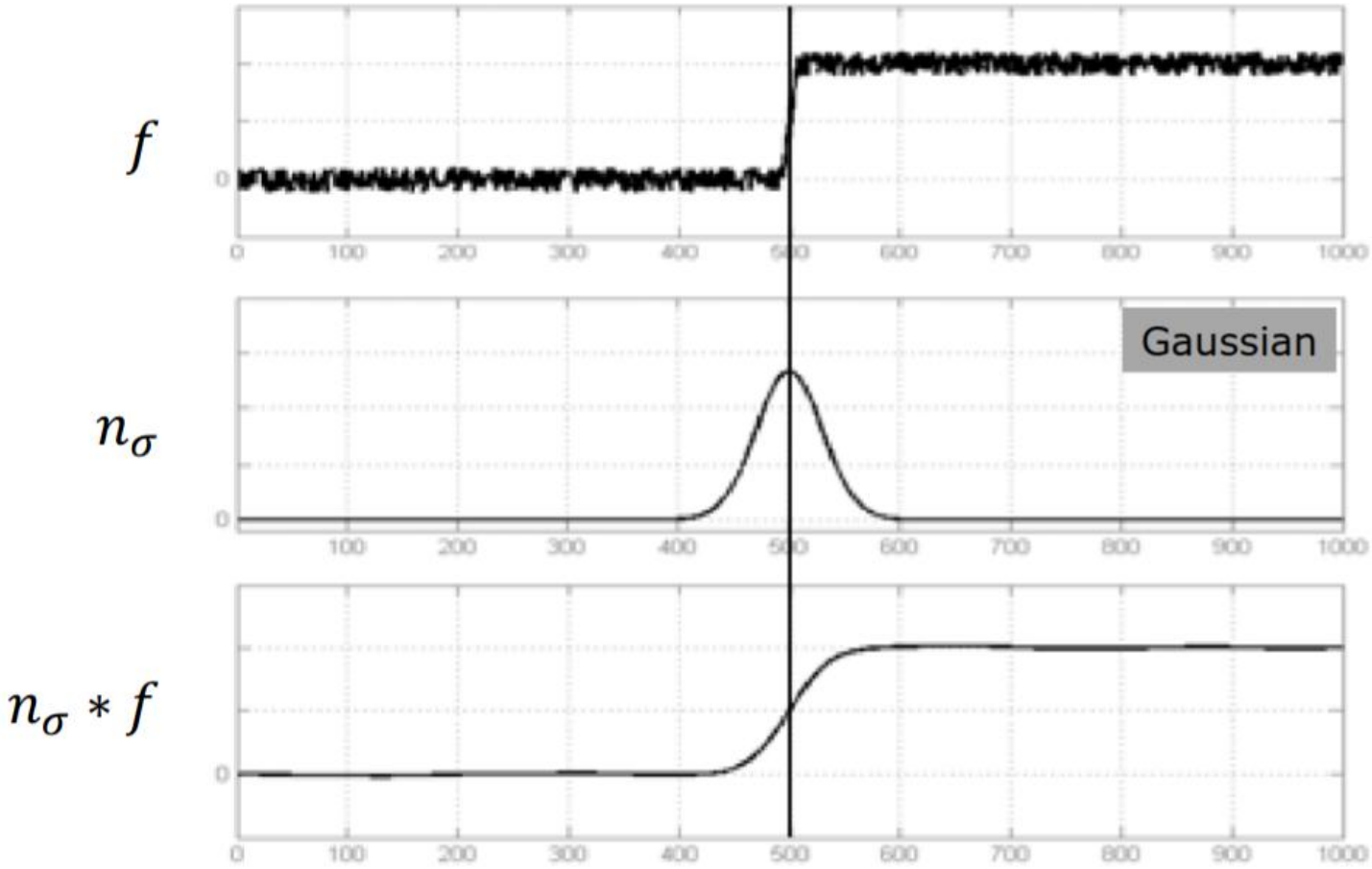
For a Blob-like Feature to be useful, we need to:

- **Locate** the blob
- Determine its **size**
- Determine its **orientation**
- Formulate a **description** or signature that is independent of size and orientation



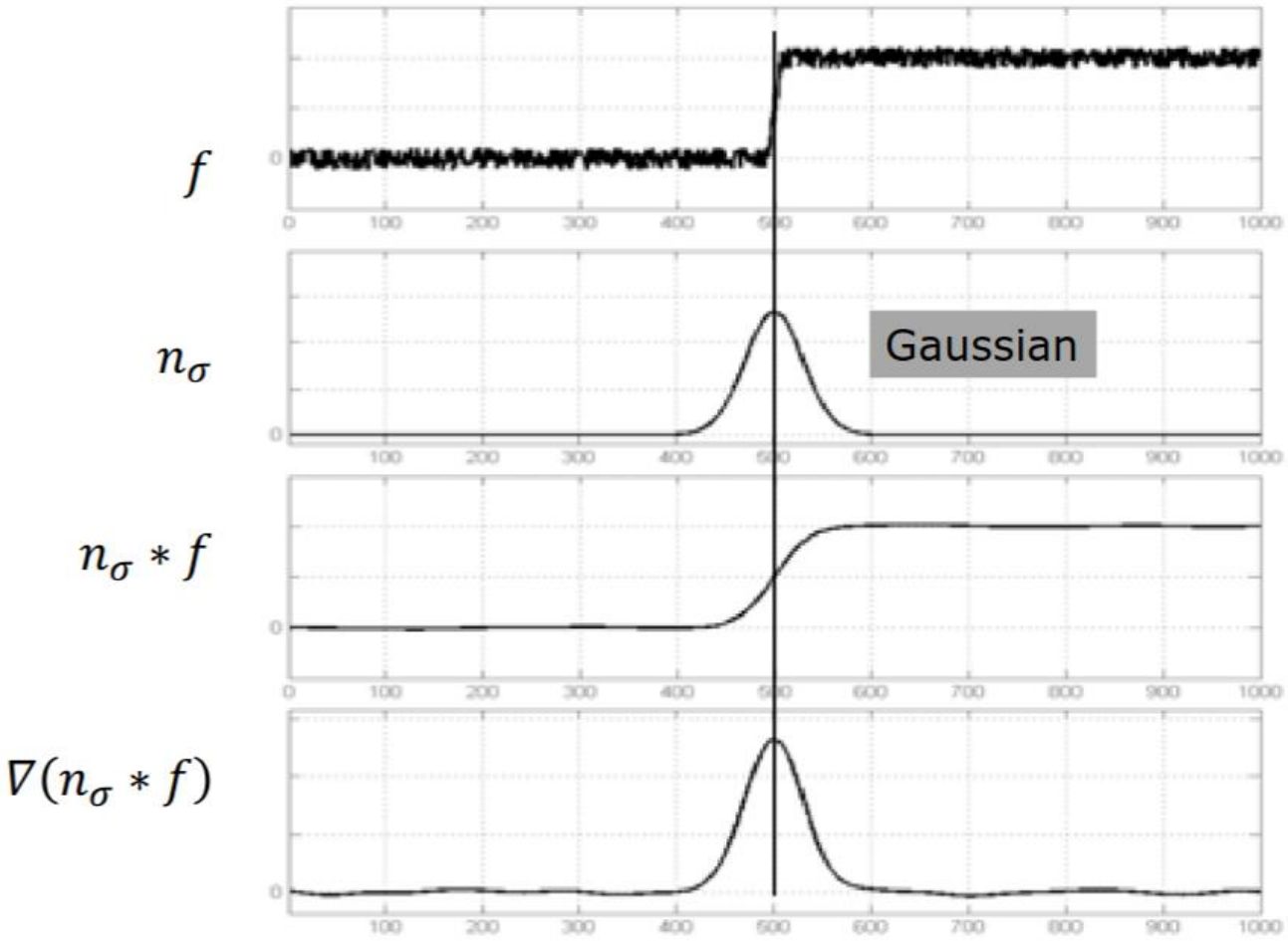
Detecting Blobs

Review: Gaussian Filter



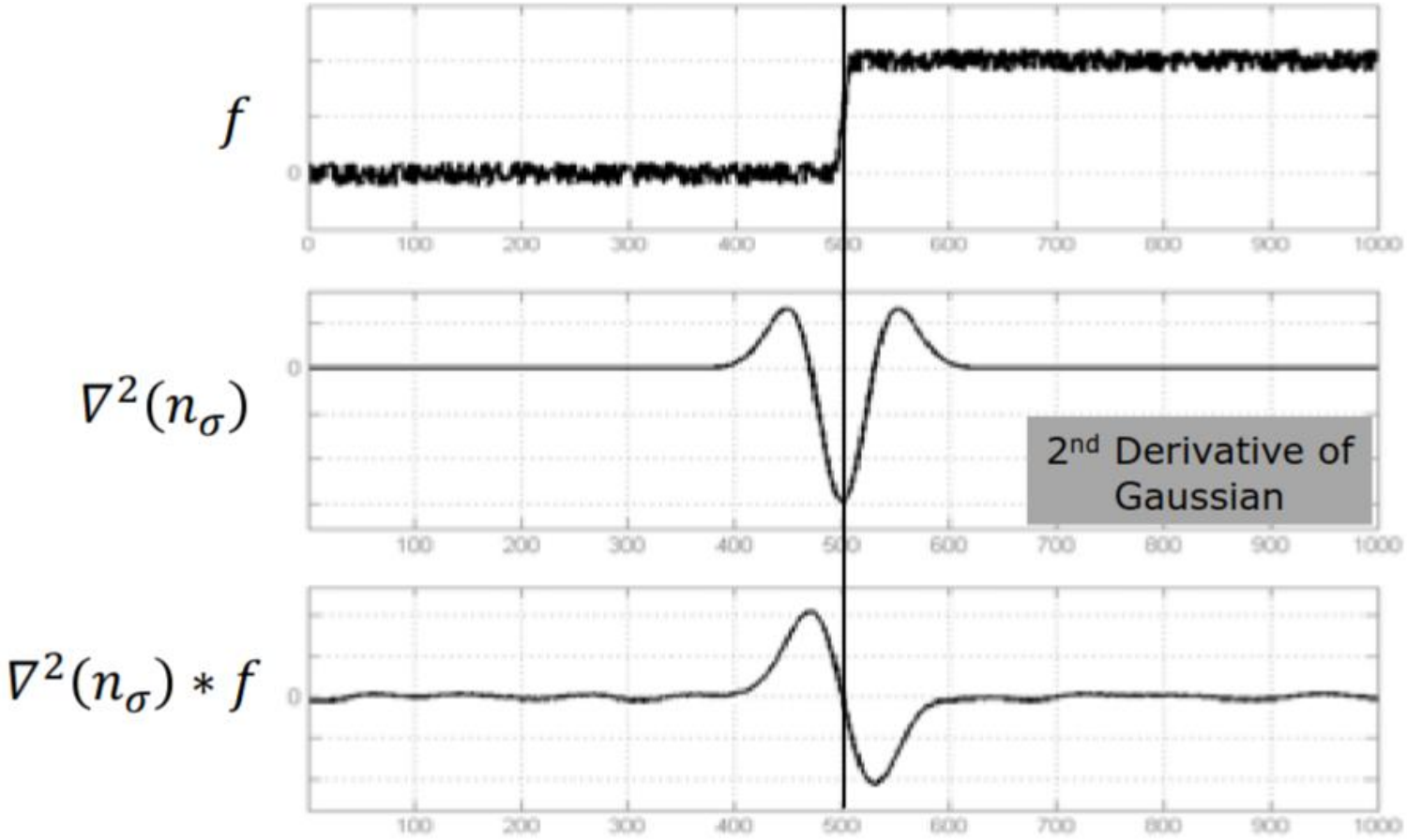
Gaussian Filter is used for removing noise by smoothing

Review: Derivative of Gaussian



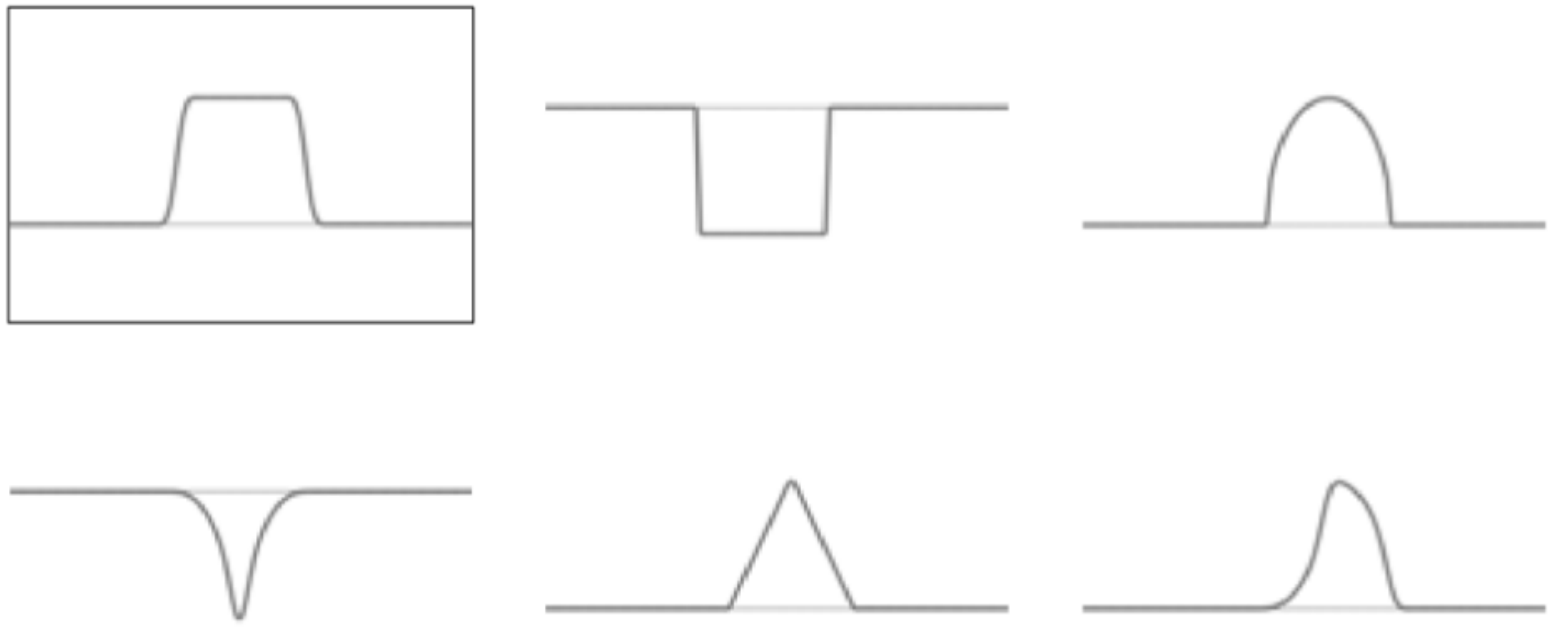
Extremum of Derivative of Gaussian denotes an Edge

Review: 2nd Derivative of Gaussian



Zero Crossing in 2nd Derivative of Gaussian denote an Edge

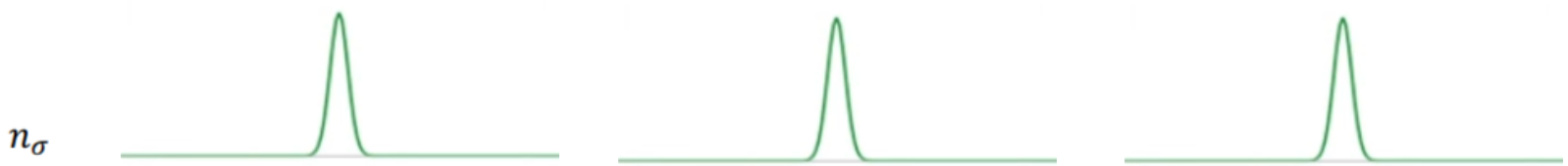
1D Blobs



Examples of 1D Blob-like structures

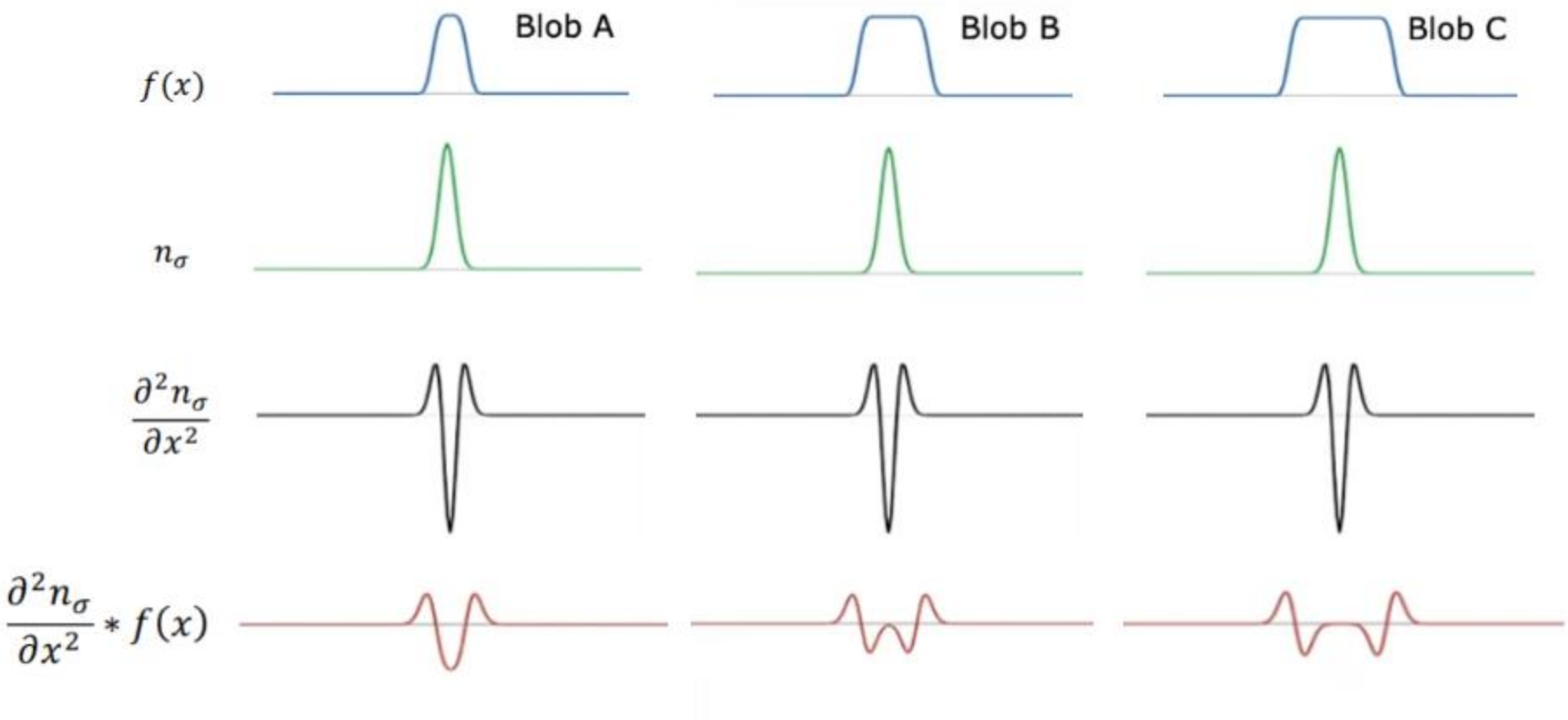
1D Blob and 2nd Derivative of Gaussian

We have three blobs, A, B, and C, where blob B is twice the width of blob A, and blob C is three times as wide as blob A.



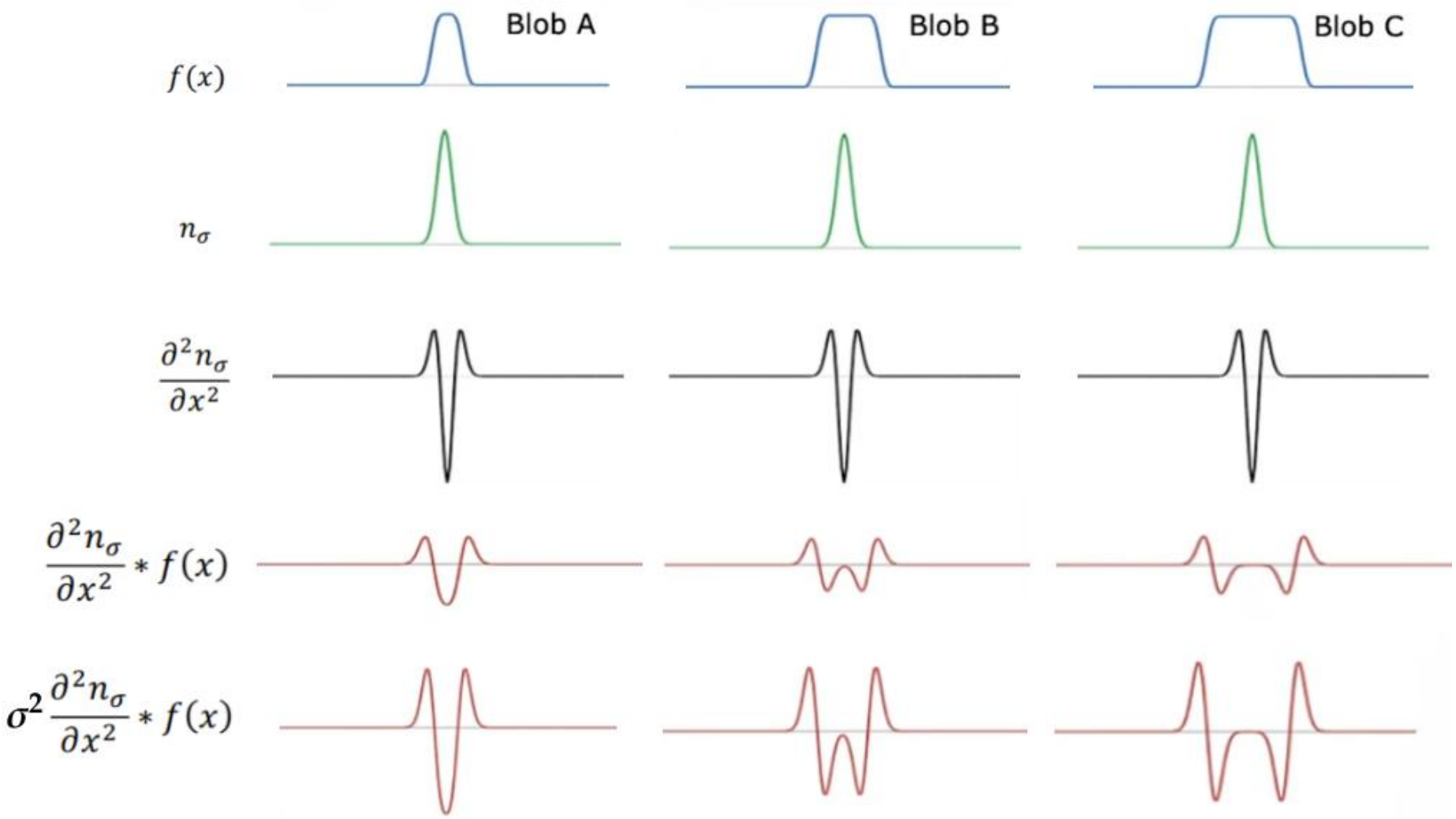
The σ (sigma) of the Gaussian is used to smooth the image and also explore what is called a "**scale space**" associated with it.

1D Blob and 2nd Derivative of Gaussian

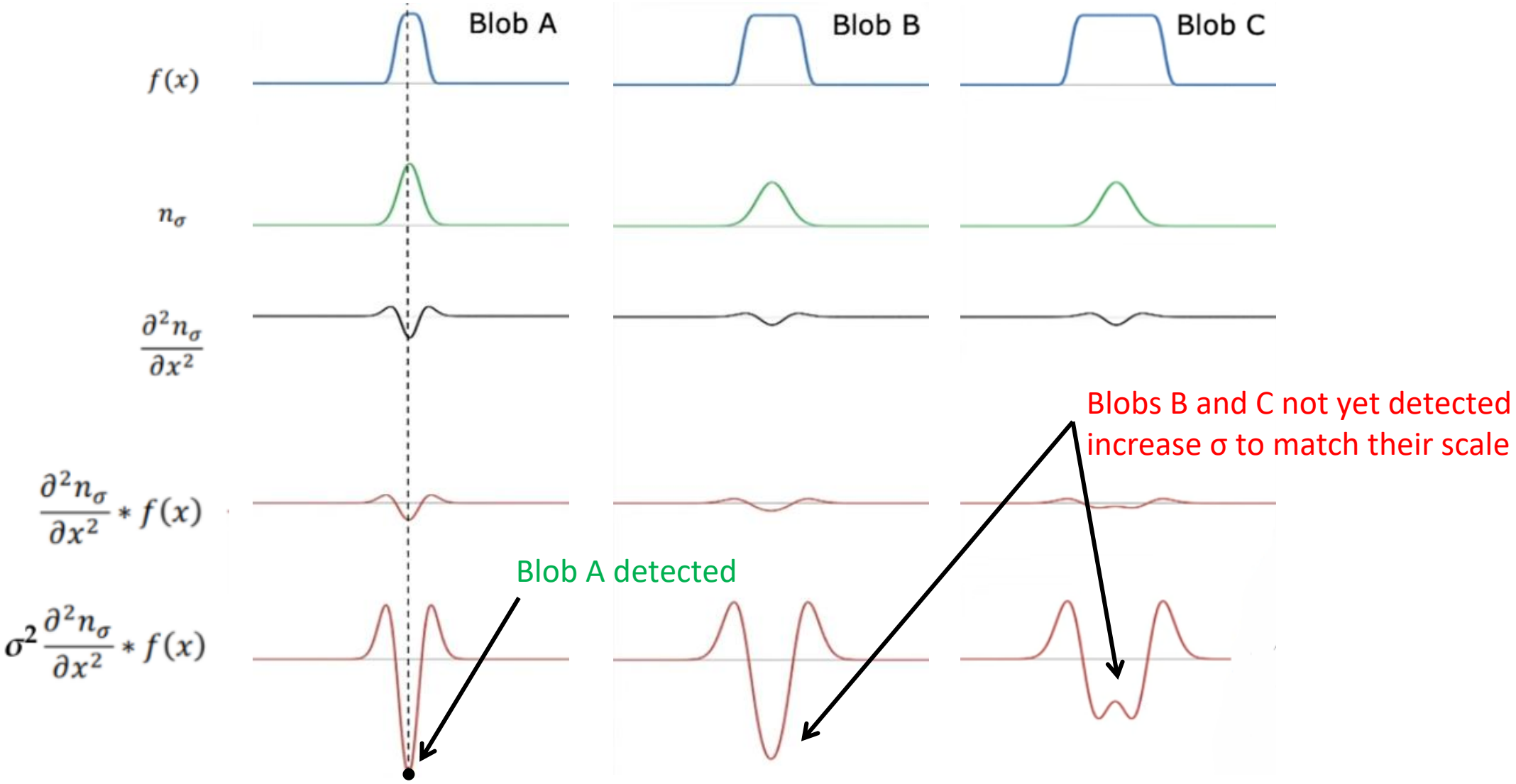


Note that the response of this operator depends on the sigma used — as sigma increases, its peak value falls, reducing the operator's response, which becomes a problem when detecting blobs at different scales.

Sigma Normalized

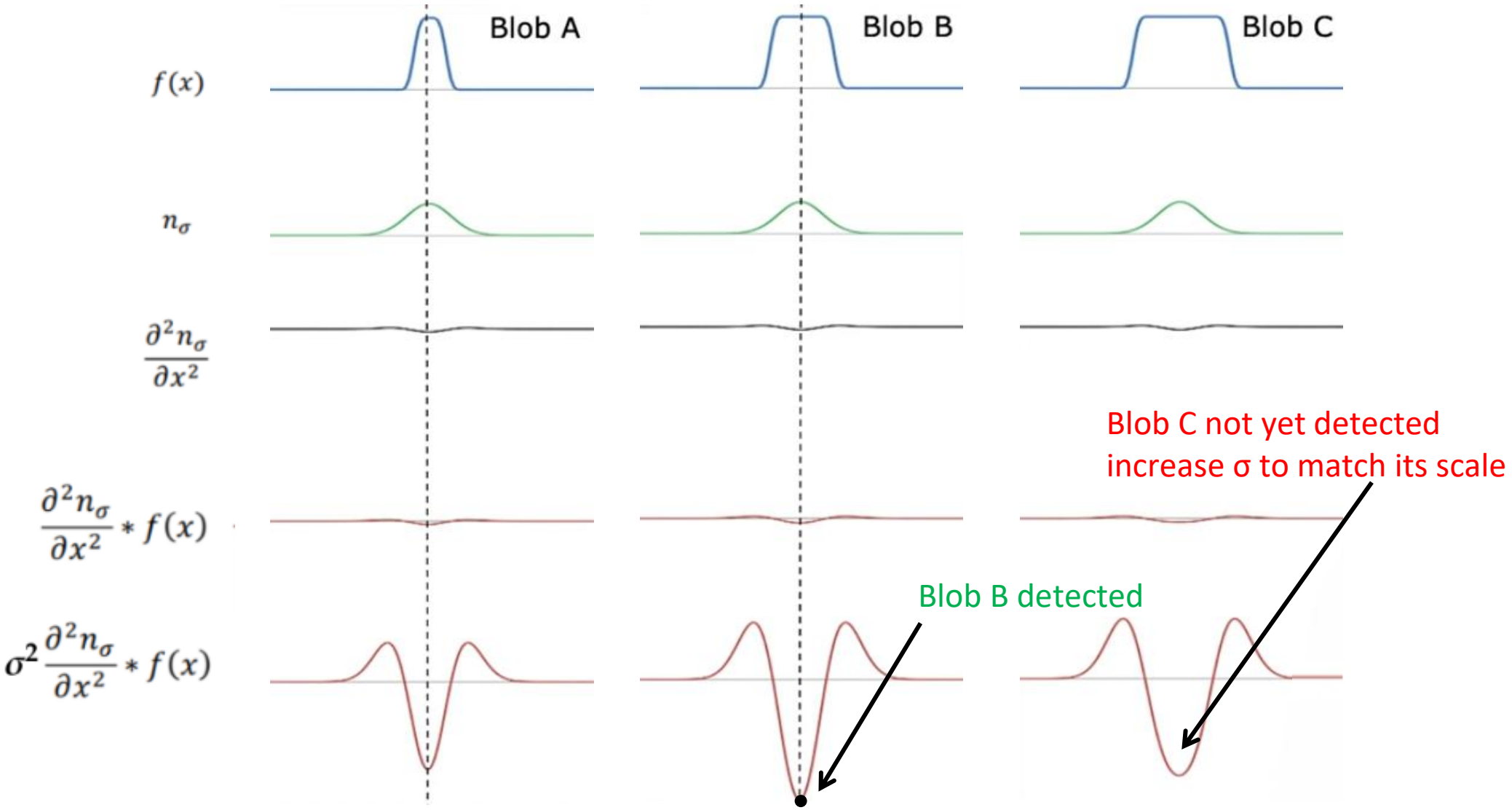


Effect of Increasing Sigma — Blob A Detected



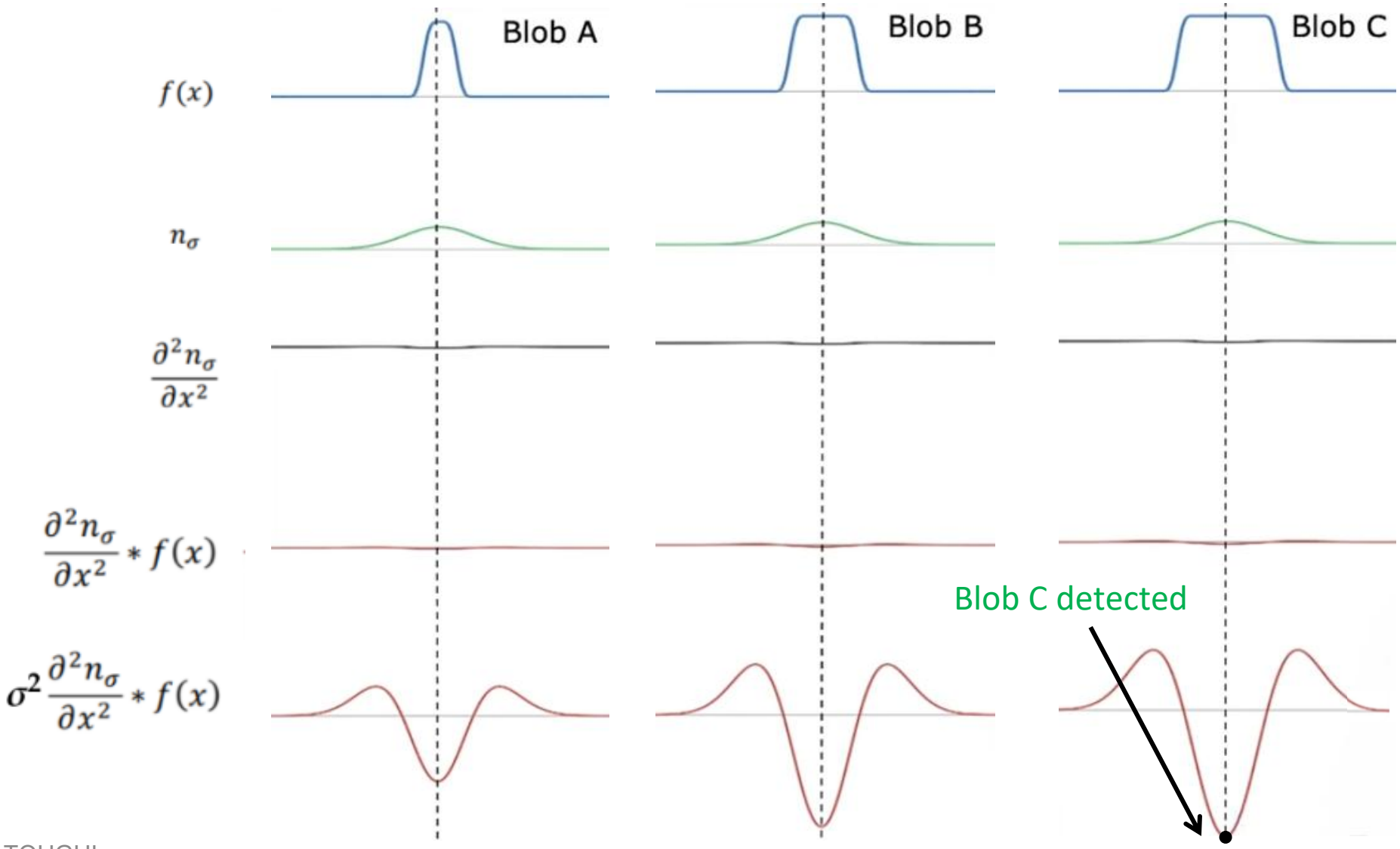
peak: local maximum in scale-normalized response

Effect of Increasing Sigma — Blob B Detected



peak: local maximum in scale-normalized response

Effect of Increasing Sigma — Blob C Detected



peak: local maximum in scale-normalized response

Detecting Blobs Across Scales

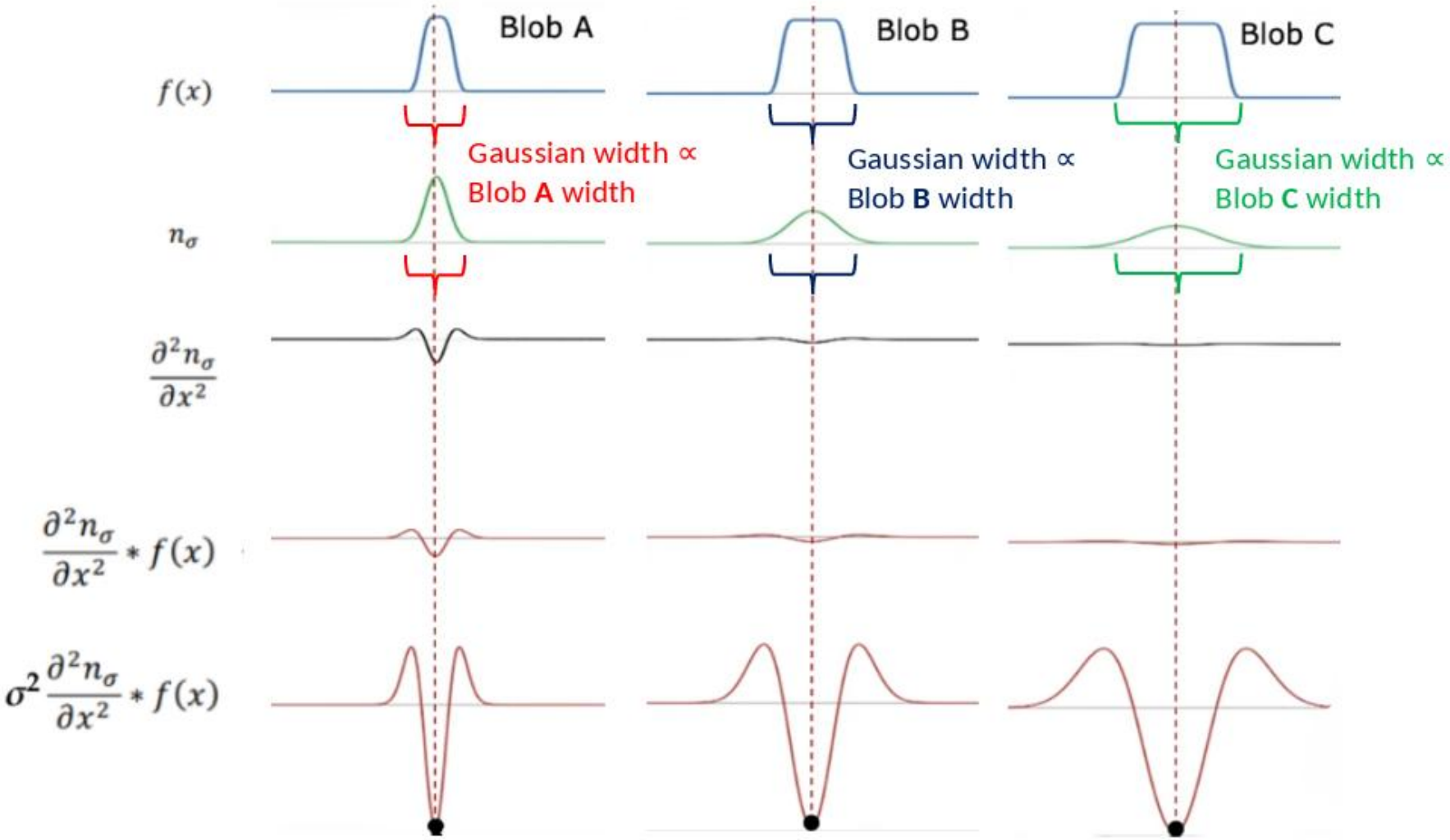
Key Takeaway: Scale-Space Blob Detection

- As σ increases, each blob produces a peak response at its matching scale — Blob A first, then B, then C.
- By applying the **scale-normalized operator** across multiple values of σ, every blob will eventually produce a local maximum — regardless of its size.

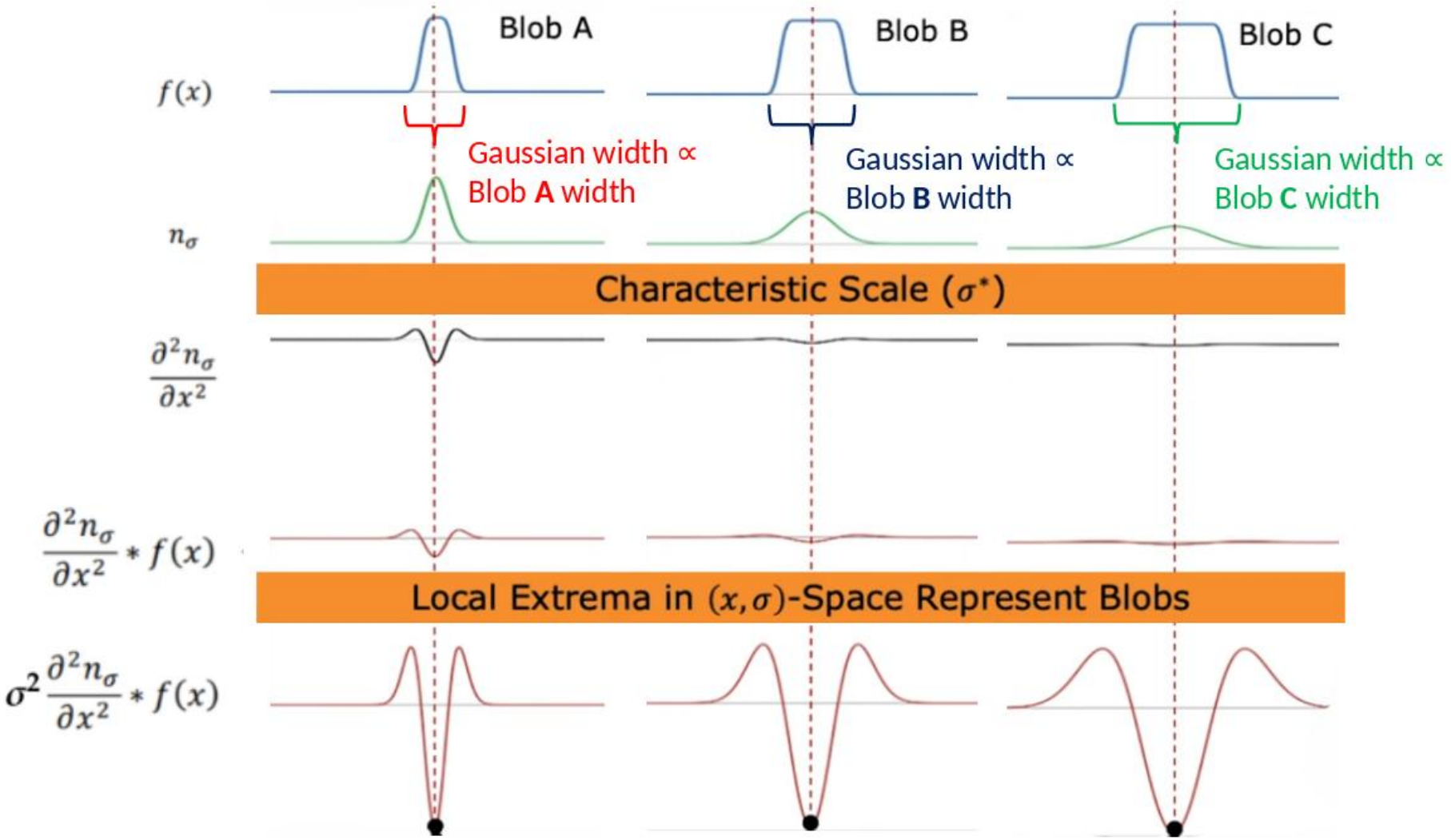
$$\sigma^2 \frac{\partial^2 \eta_\sigma}{\partial x^2} * f(x)$$

- σ is referred to as **scale** — and the location + scale of each peak gives us both the position and size of the detected blob.

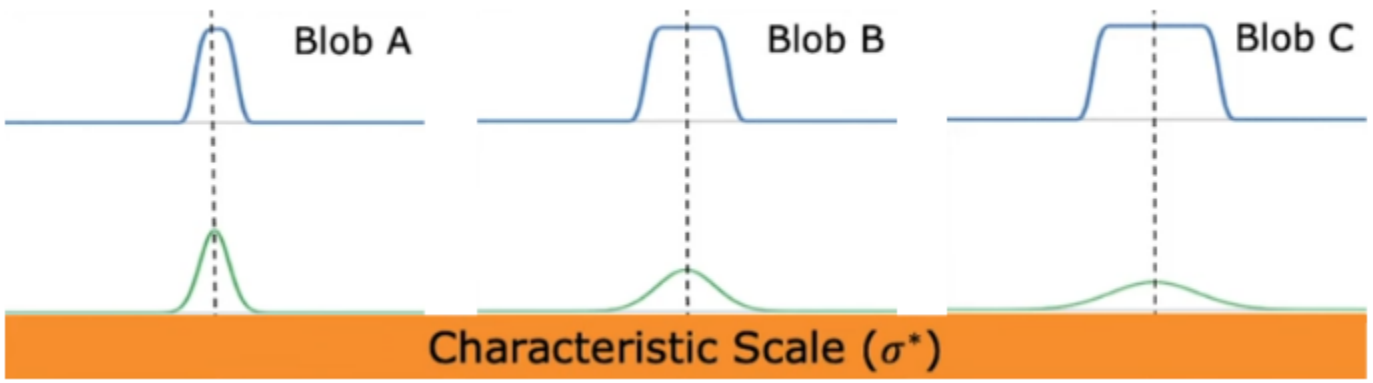
Detecting Blobs Across Scales



Detecting Blobs Across Scales



Detecting Blobs Across Scales



$\sigma_A{}^* = \sigma_1$ $\sigma_B{}^* = 2\sigma_1$ $\sigma_C{}^* = 3\sigma_1$

Characteristic Scale: The σ at which σ-normalized 2nd derivative attains its extreme value.

$$\boxed{\text{Characteristic Scale} \propto \text{Size of Blob}}$$

$$\frac{\text{Size of Blob A}}{\text{Size of Blob B}} = \frac{\sigma_A{}^*}{\sigma_B{}^*} \; ; \quad \frac{\text{Size of Blob B}}{\text{Size of Blob C}} = \frac{\sigma_B{}^*}{\sigma_C{}^*}$$

1D Blob Detection Summary

Given: 1D signal $f(x)$

Compute: $\sigma^2 \frac{\partial^2 n_\sigma}{\partial x^2} * f(x)$ at many scales $(\sigma_0, \sigma_1, \sigma_2, \ldots, \sigma_k)$.

Find: $$\boxed{(x^*, \sigma^*) = \arg\max_{(x,\sigma)} \left| \sigma^2 \frac{\partial^2 n_\sigma}{\partial x^2} * f(x) \right|}$$

x^*: Blob Position

σ^*: Characteristic Scale (Blob Size)

2D Blob Detector

Normalized Laplacian of Gaussian (NLoG) is used as the 2D equivalent for Blob Detection

Laplacian

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

Gaussian

n_σ

LoG

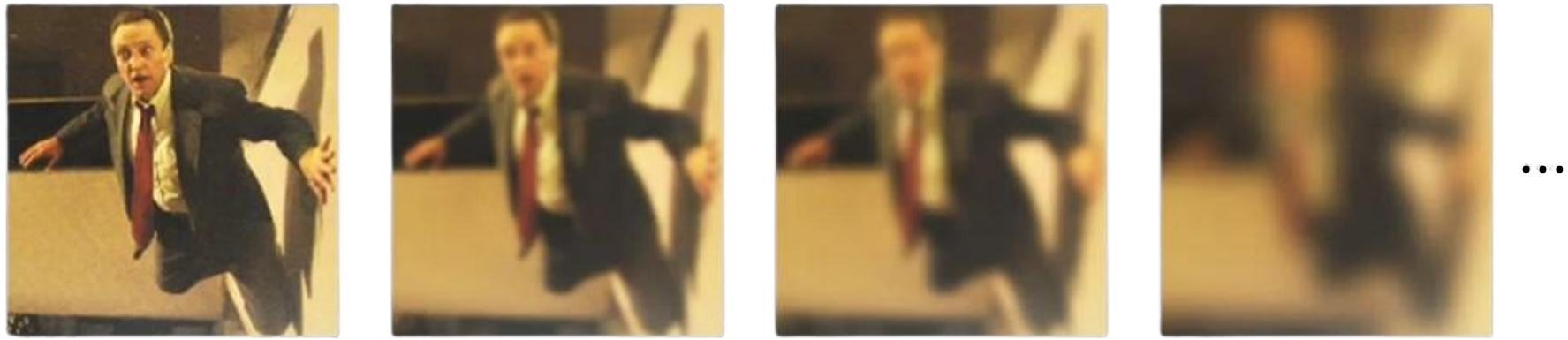
$\nabla^2 n_\sigma$

NLoG

$\sigma^2 \nabla^2 n_\sigma$

Location of Blobs given by Local Extrema after applying Normalized Laplacian of Gaussian at many scales.

Scale-Space



$S(x, y, \sigma_0)$ $S(x, y, \sigma_1)$ $S(x, y, \sigma_2)$ $S(x, y, \sigma_3)$ ...

Increasing σ, Higher Scale, Lower Resolution

Scale Space: Stack created by filtering an image with Gaussians of different sigma (σ)

$$S(x, y, \sigma) = n(x, y, \sigma) * I(x, y)$$

Creating Scale-Space

$S(x, y, \sigma_0)$ $S(x, y, \sigma_1)$ $S(x, y, \sigma_2)$ $S(x, y, \sigma_3)$ …

Increasing σ, Higher Scale, Lower Resolution

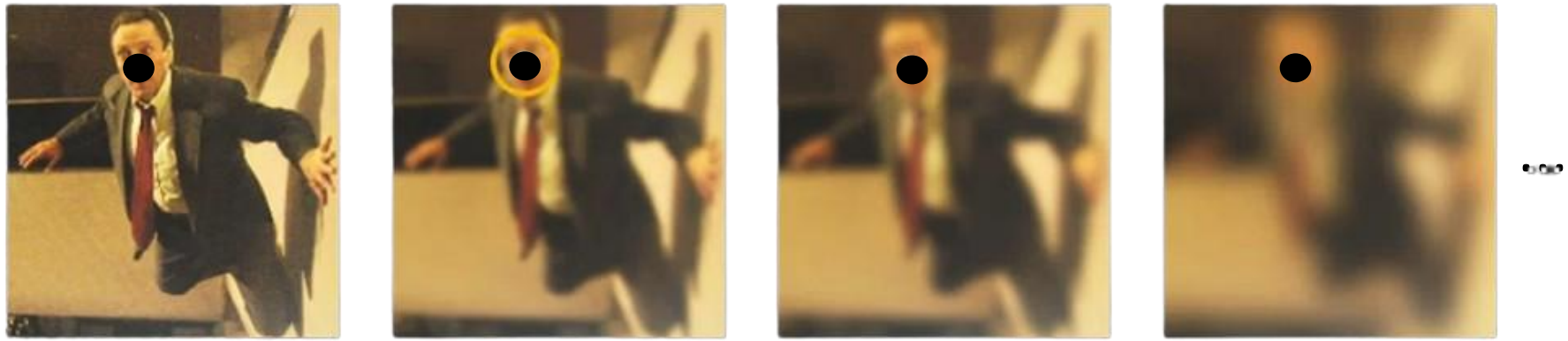
Selecting sigmas to generate the scale-space:

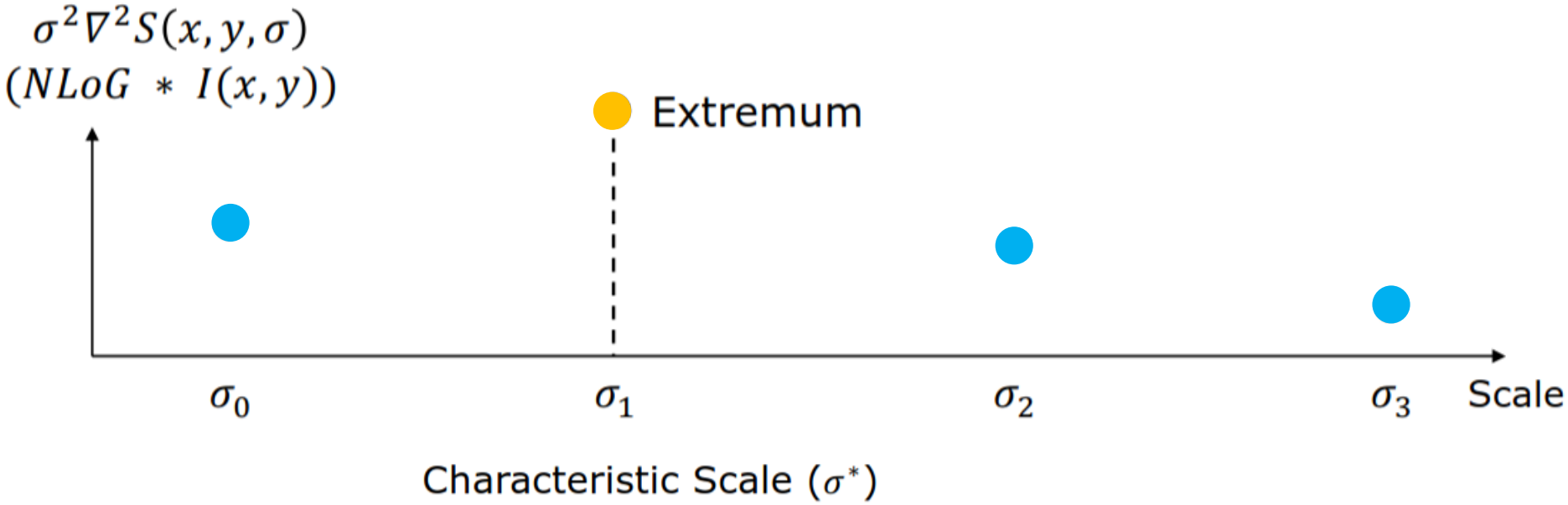
$$\boxed{\sigma_k = \sigma_0 s^k} \quad k = 0,1,2,3, \ldots$$

s: Constant multiplier

σ_0: Initial Scale

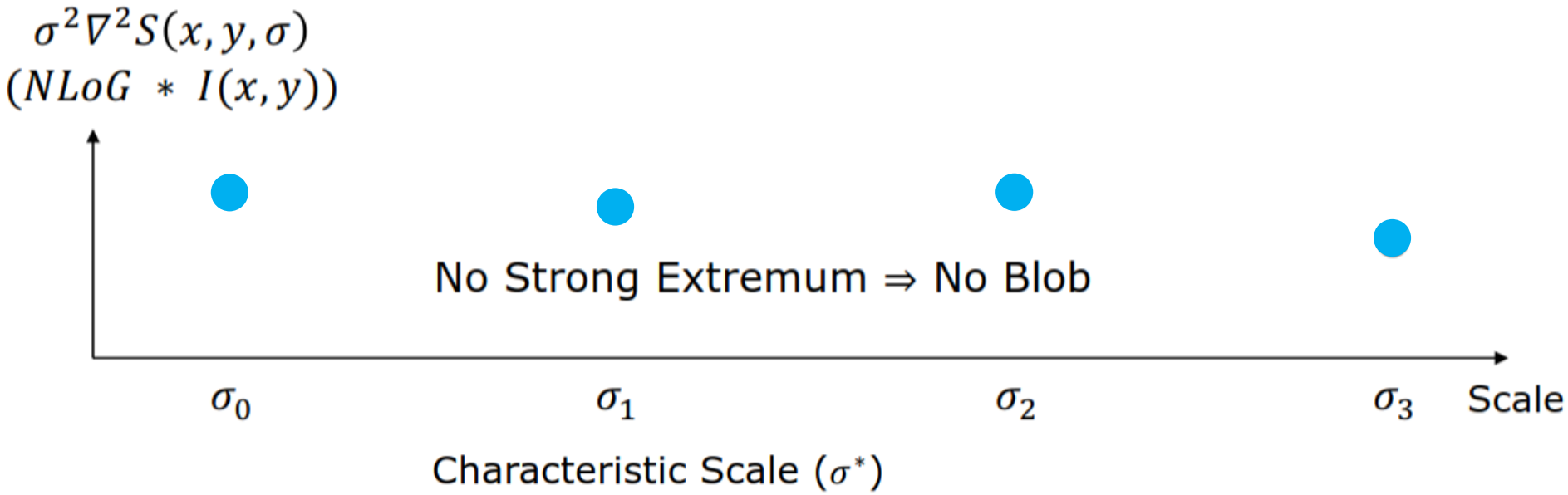
Creating Scale-Space





Creating Scale-Space

...



2D Blob Detection Summary

Given an image $I(x, y)$

Convolve the image using NLoG at many scales σ

Find:

$$\boxed{(x^*, y^*, \sigma^*) = \underset{(x,y,\sigma)}{\arg\max} \; |\sigma^2 \nabla^2 n_\sigma * I(x, y)|}$$

(x^*, y^*): Position of the blob

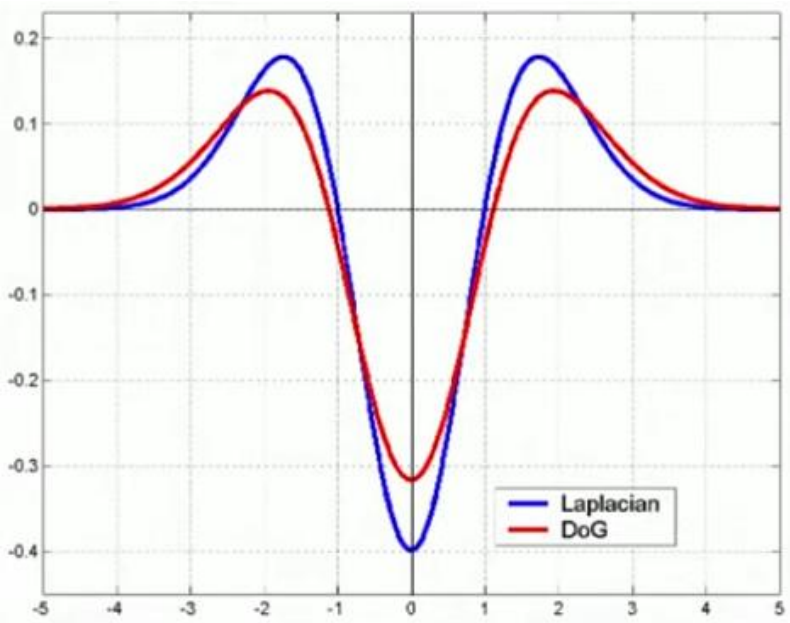
σ^*: Size of the blob

SIFT Detector

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. International journal of computer vision, 60(2), 91-110.

Fast NLoG Approximation: DoG

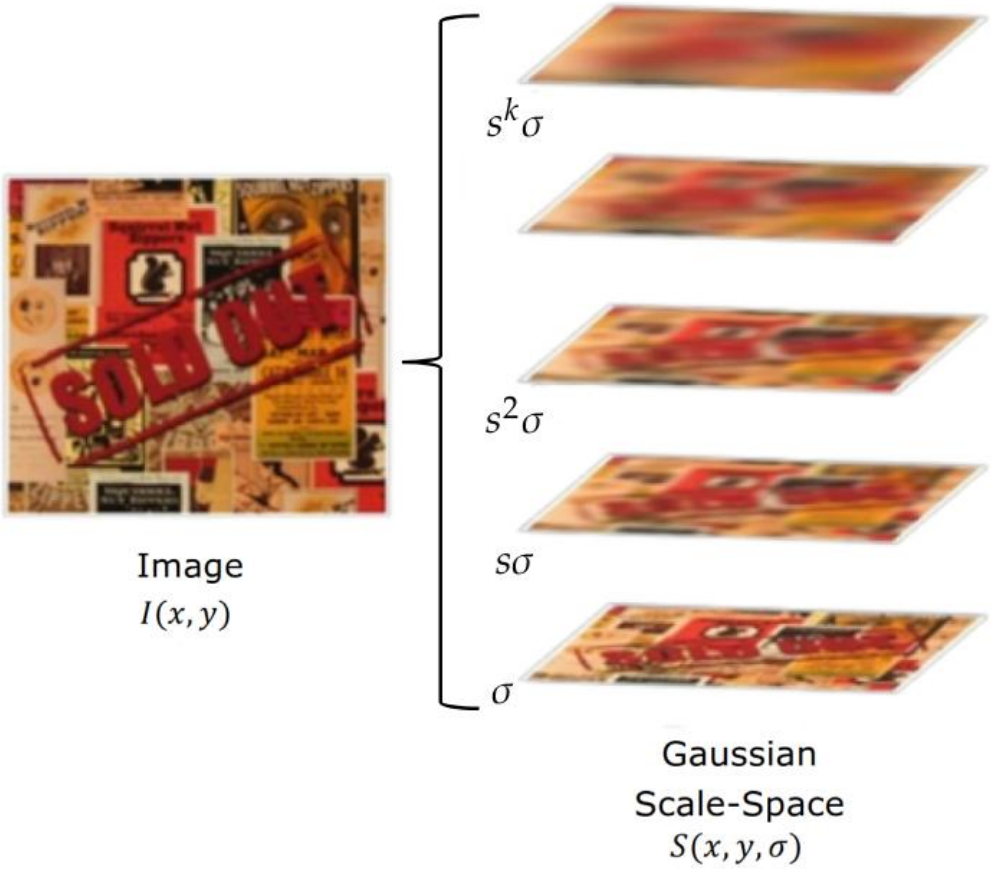
$$\text{Difference of Gaussian (DoG)} = (n_{s\sigma} - n_{\sigma}) \approx (s-1)\underbrace{\sigma^2 \nabla^2 n_{\sigma}}_{\text{NLoG}}$$



$$\boxed{\text{DoG} \approx (s-1)\ \text{NLoG}}$$

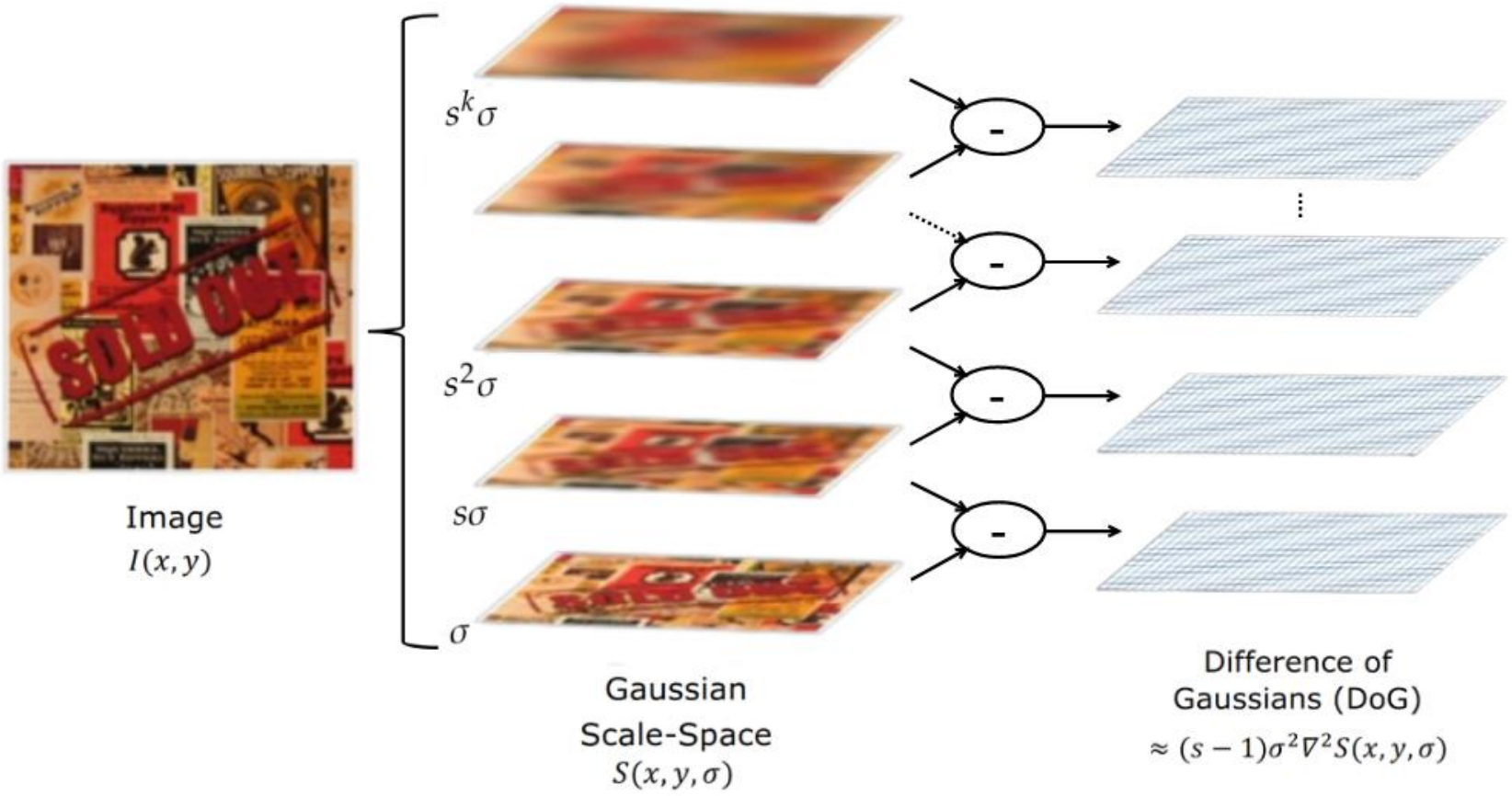
The DoG is a computationally efficient approximation of the normalized Laplacian of Gaussian — **simply subtracting two consecutive smoothed images gives the same result, without explicitly computing the Laplacian.**

Extracting SIFT Interest Points



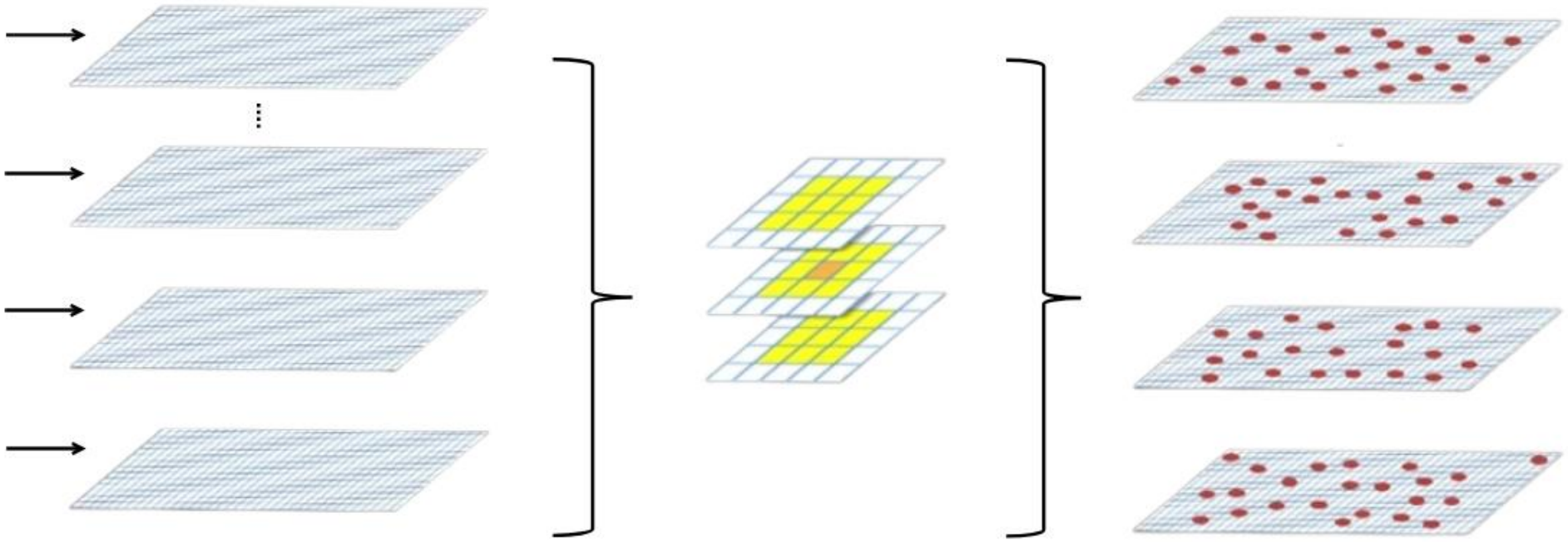
The first step in SIFT is building a scale space — a stack of images created by smoothing the original image with increasing values of sigma.

Extracting SIFT Interest Points



By subtracting consecutive images in the scale space stack, we obtain a new stack of Difference of Gaussians — each representing the normalized Laplacian of Gaussian at a different scale.

Extracting SIFT Interest Points



Difference of Gaussians (DoG)

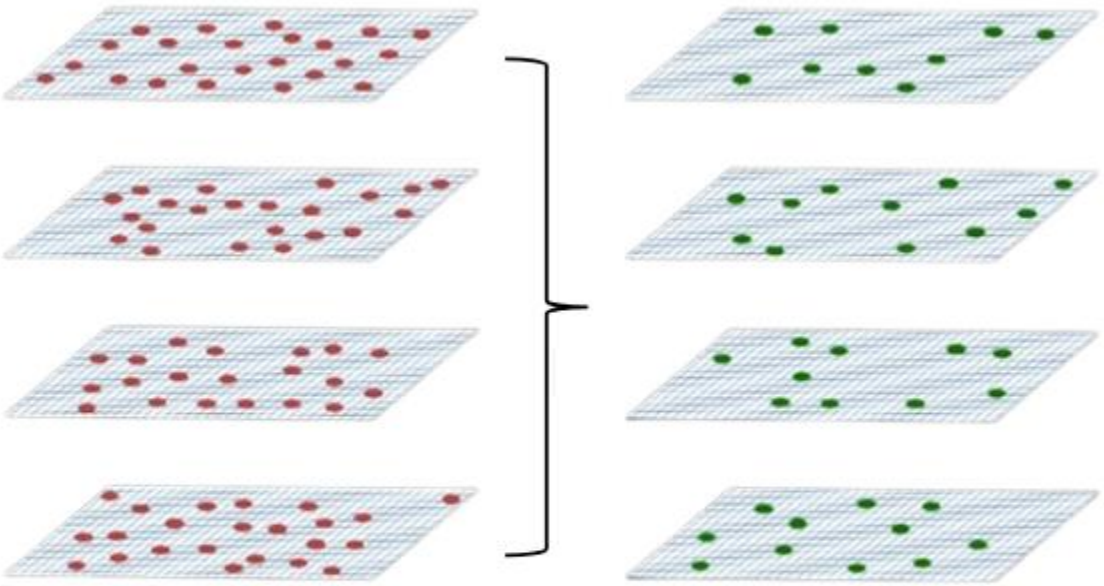
$\approx (s-1)\sigma^2\nabla^2 S(x,y,\sigma)$

Find Extremum in every 3x3x3 grid

Interest Point Candidates

(includes weak extrema)

Extracting SIFT Interest Points



Interest Point Candidates
(includes weak extrema)

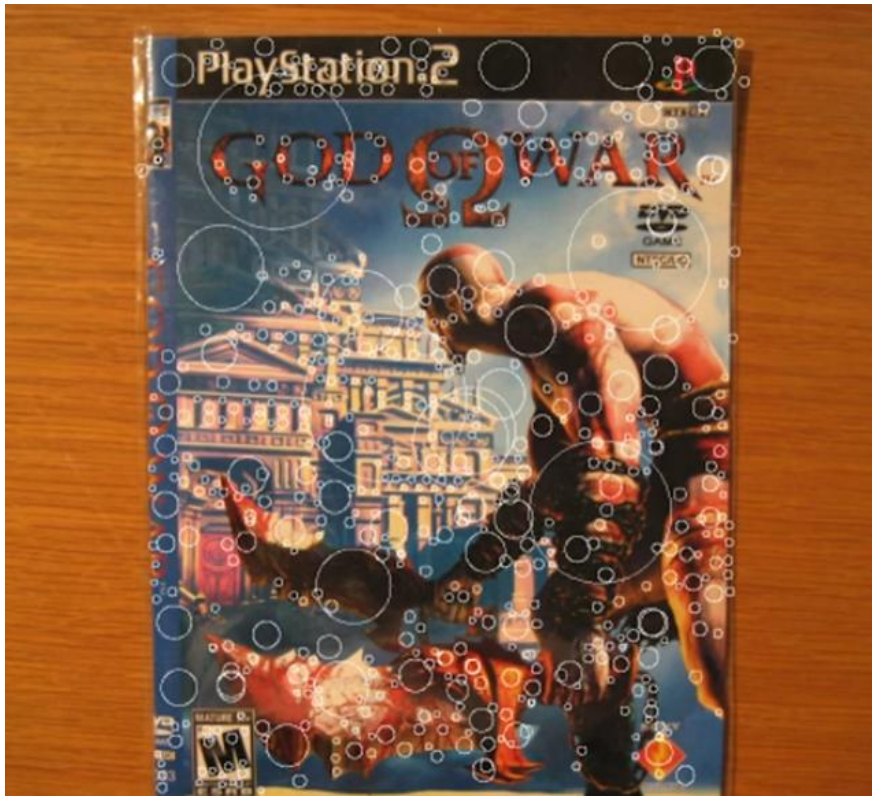
SIFT Interest Points
(after removing weak extrema)

Interest Point Visualization

Each interest point has:

- A **position** in the image
- A **scale** — blob size is proportional to the scale at which it was detected

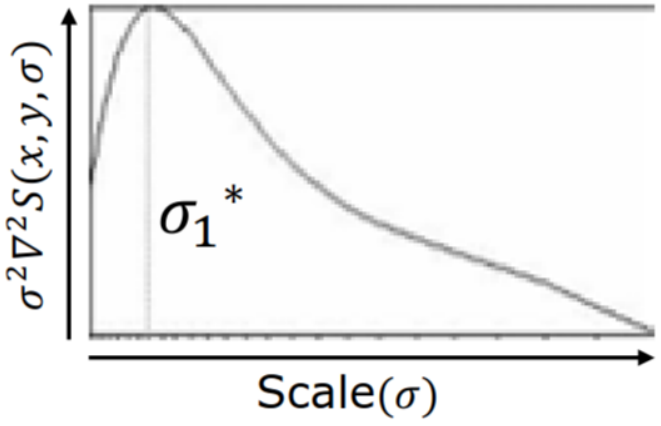
SIFT Detection Examples

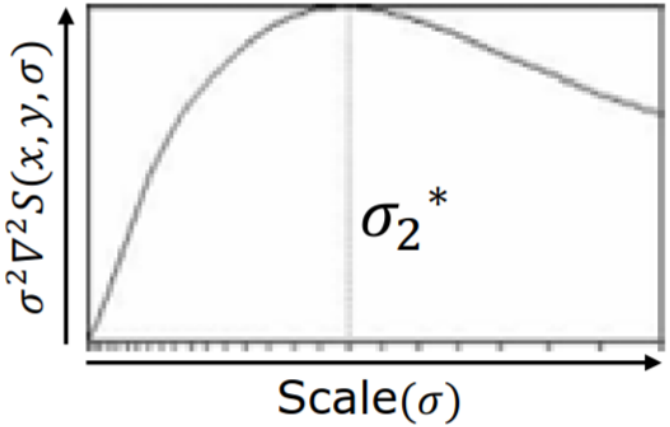


SIFT detects blob-like regions with distinctive local appearance — not necessarily obvious blobs, but patches interesting enough to be reliably matched across images.

SIFT Scale Invariance

SIFT feature



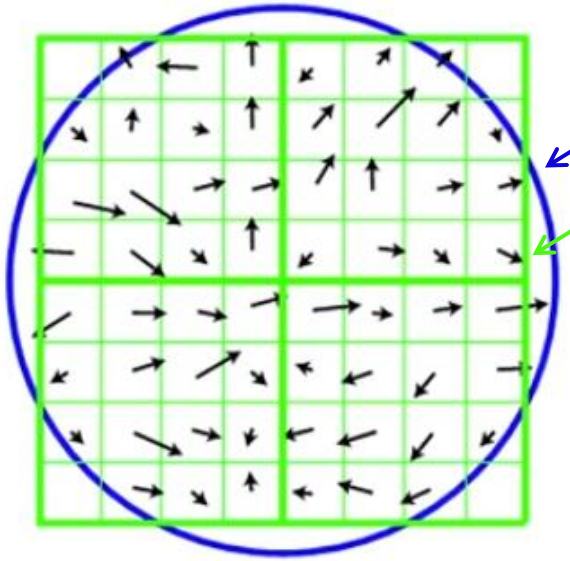


SIFT Scale Invariance

Since the same feature peaks at different sigmas depending on scale, we can use the known sigma values to normalize and align the two features — making SIFT scale invariant.

$$\frac{\sigma_1{}^*}{\sigma_2{}^*}\text{: Ratio of Blob Sizes}$$

Computing the Principal Orientation



blob area

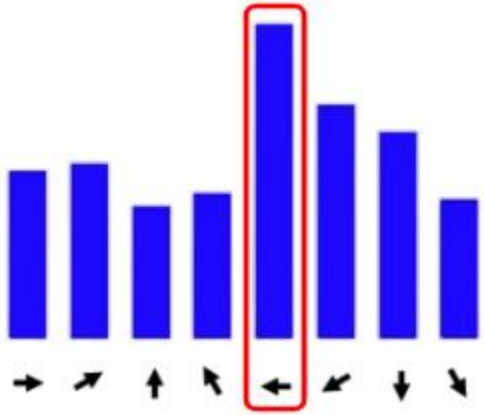
a grid of edge directions

Image gradient directions

$$\theta = \tan^{-1}\left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x}\right)$$

Choose the most prominent gradient direction

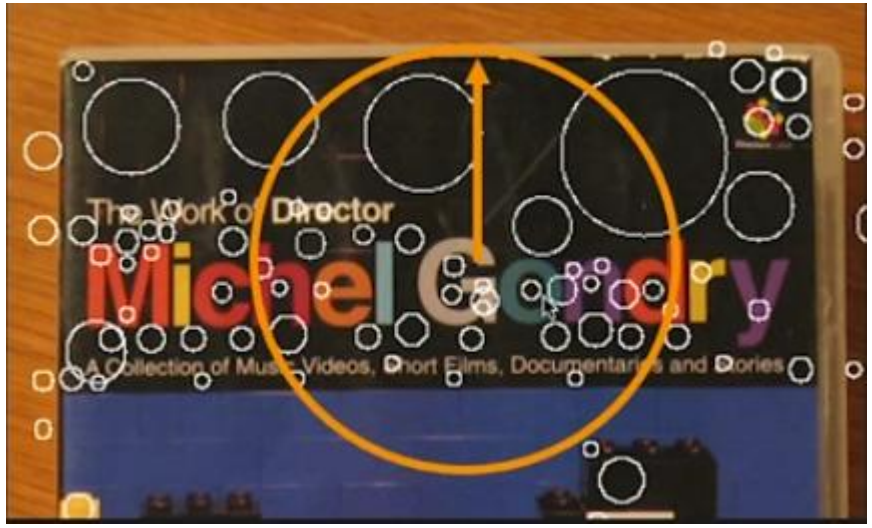
Principal Orientation

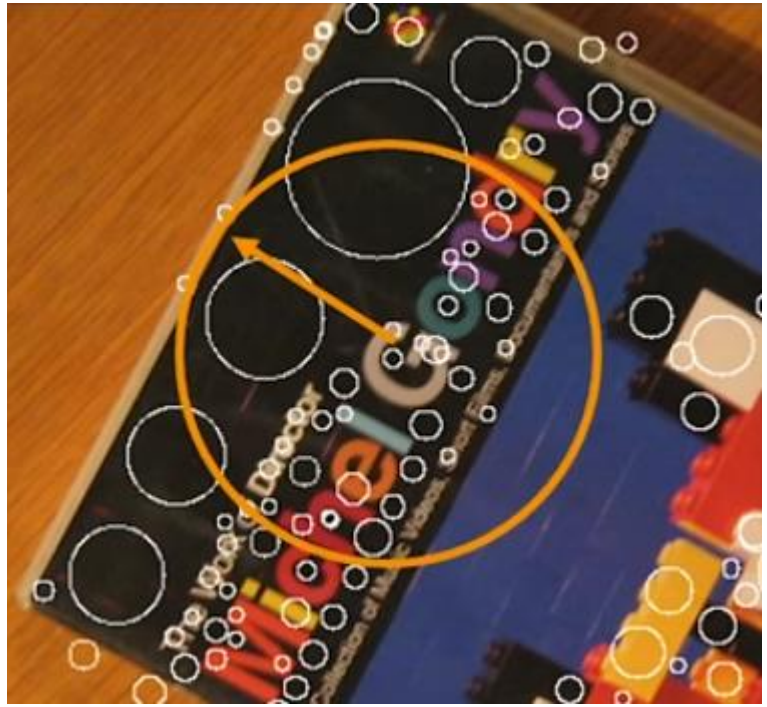


To achieve rotation invariance, we compute a histogram of gradient orientations within the blob region — **ignoring magnitude to reduce sensitivity to lighting** — and define the peak of this histogram as the principal orientation of the feature.

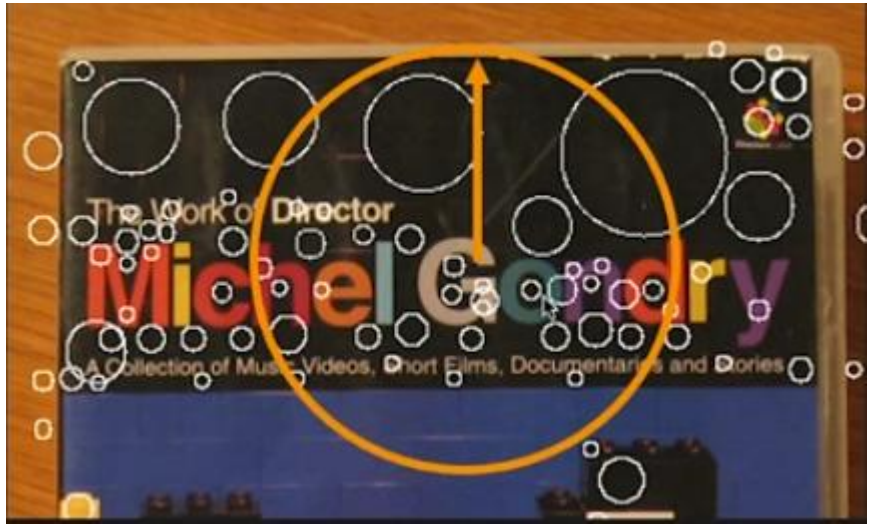
SIFT Rotation Invariance

Use the principal orientation to undo rotation





SIFT Rotation Invariance



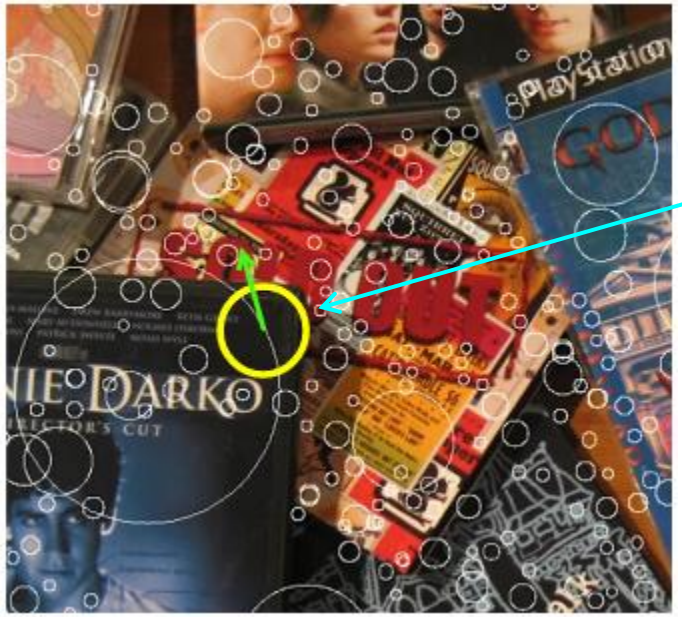
By computing the principal orientation for each feature, we can undo the effect of rotation during matching — combined with scale normalization, SIFT is fully invariant to both scale and rotation.

SIFT Descriptor

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. International journal of computer vision, 60(2), 91-110.

SIFT Descriptor

Before normalization



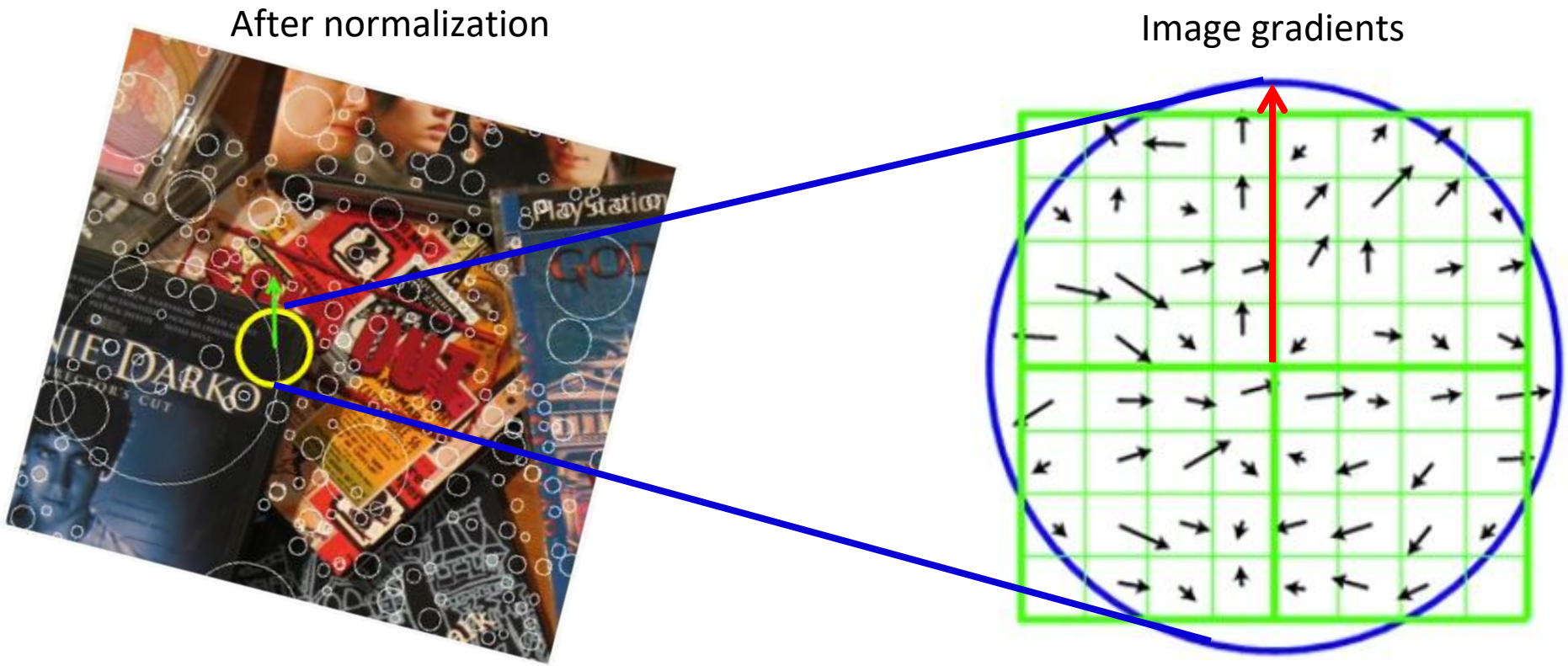
SIFT feature

Normalize scale
& rotation

After normalization

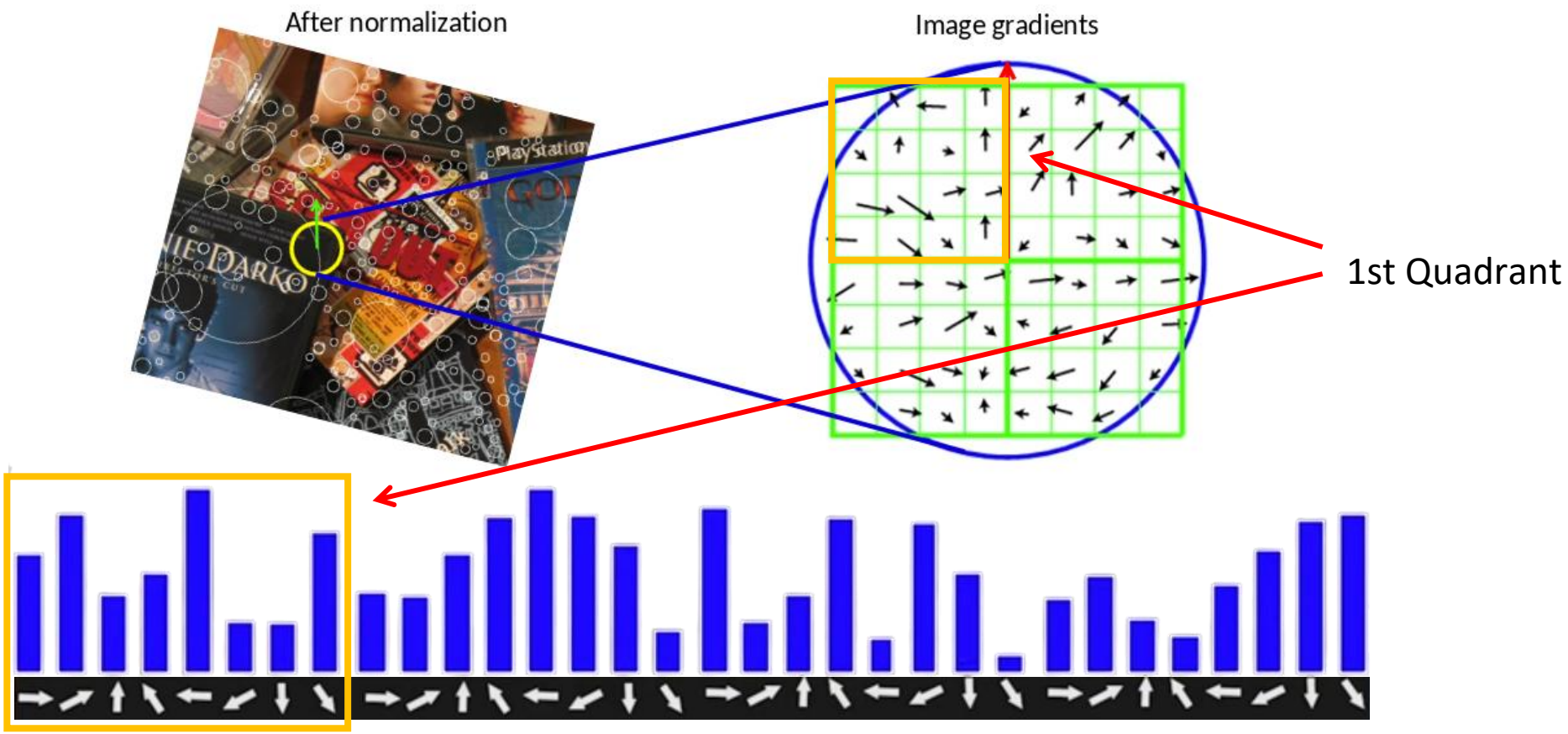
Once a SIFT interest point is detected, we normalize it by undoing both its scale and rotation — preparing it for descriptor extraction.

SIFT Descriptor



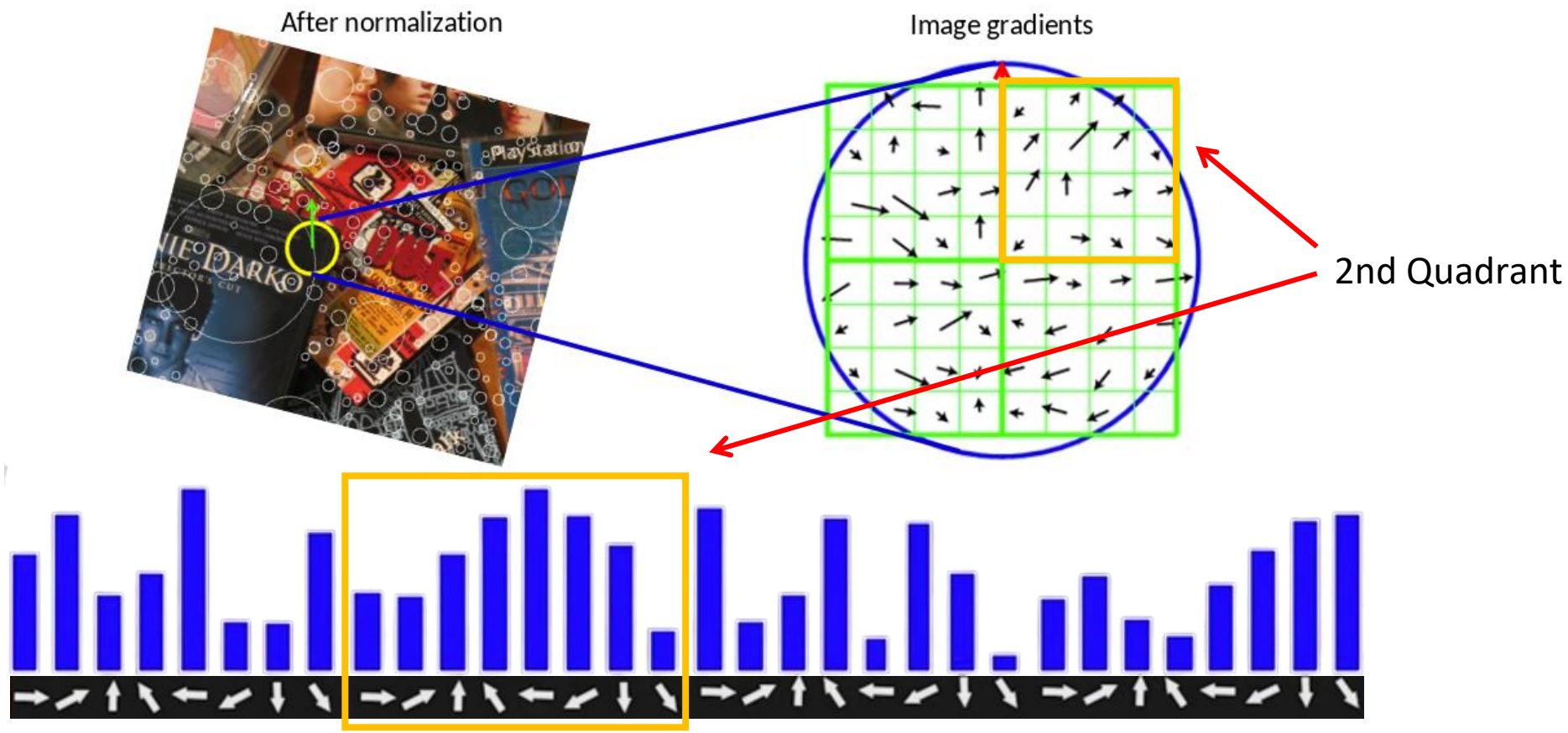
The normalized feature is placed on a standard grid, and for each pixel we compute the gradient orientation only — ignoring magnitude to remain robust to lighting changes.

SIFT Descriptor



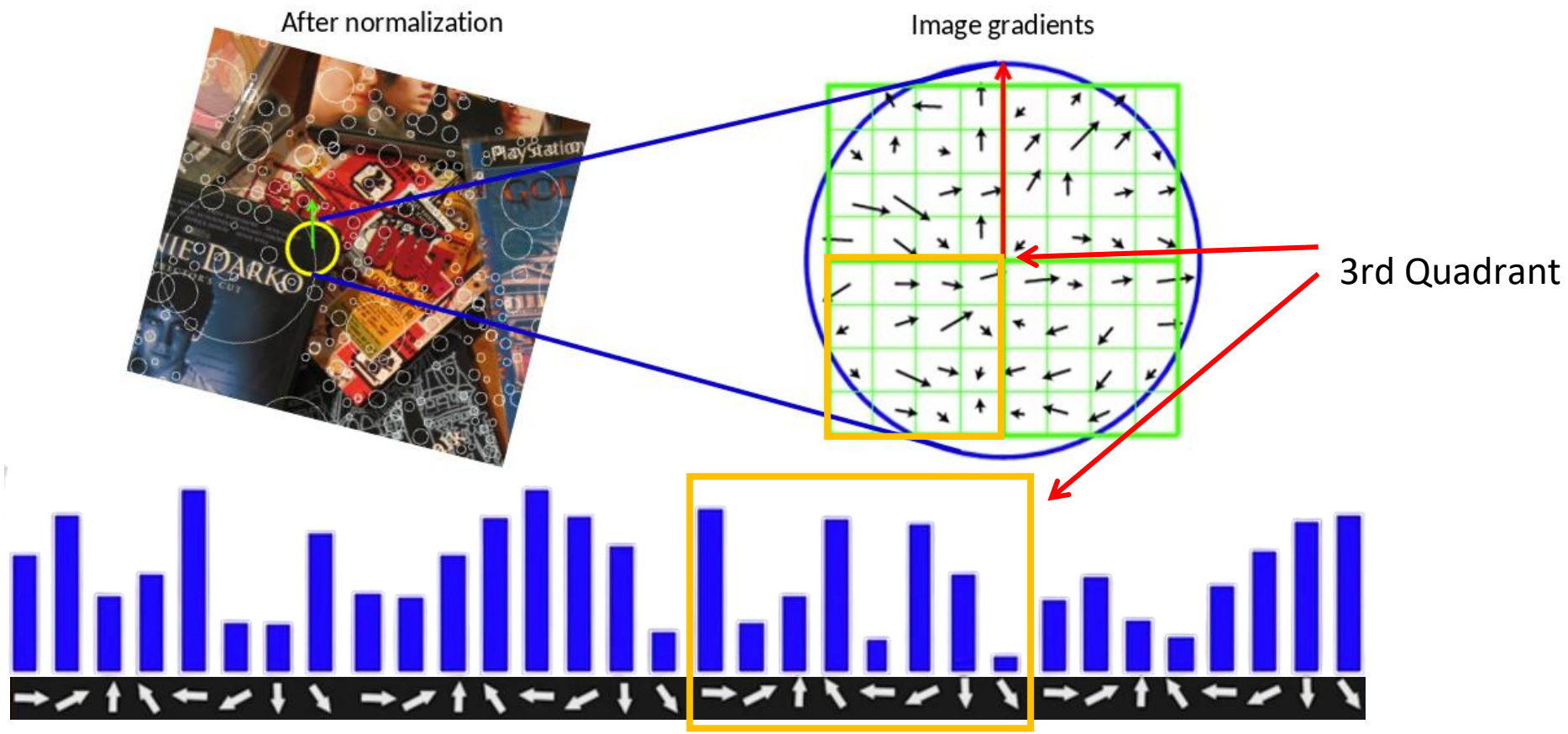
The grid is divided into quadrants, and for each quadrant we compute a gradient orientation histogram — counting how many pixels point in each direction.

SIFT Descriptor



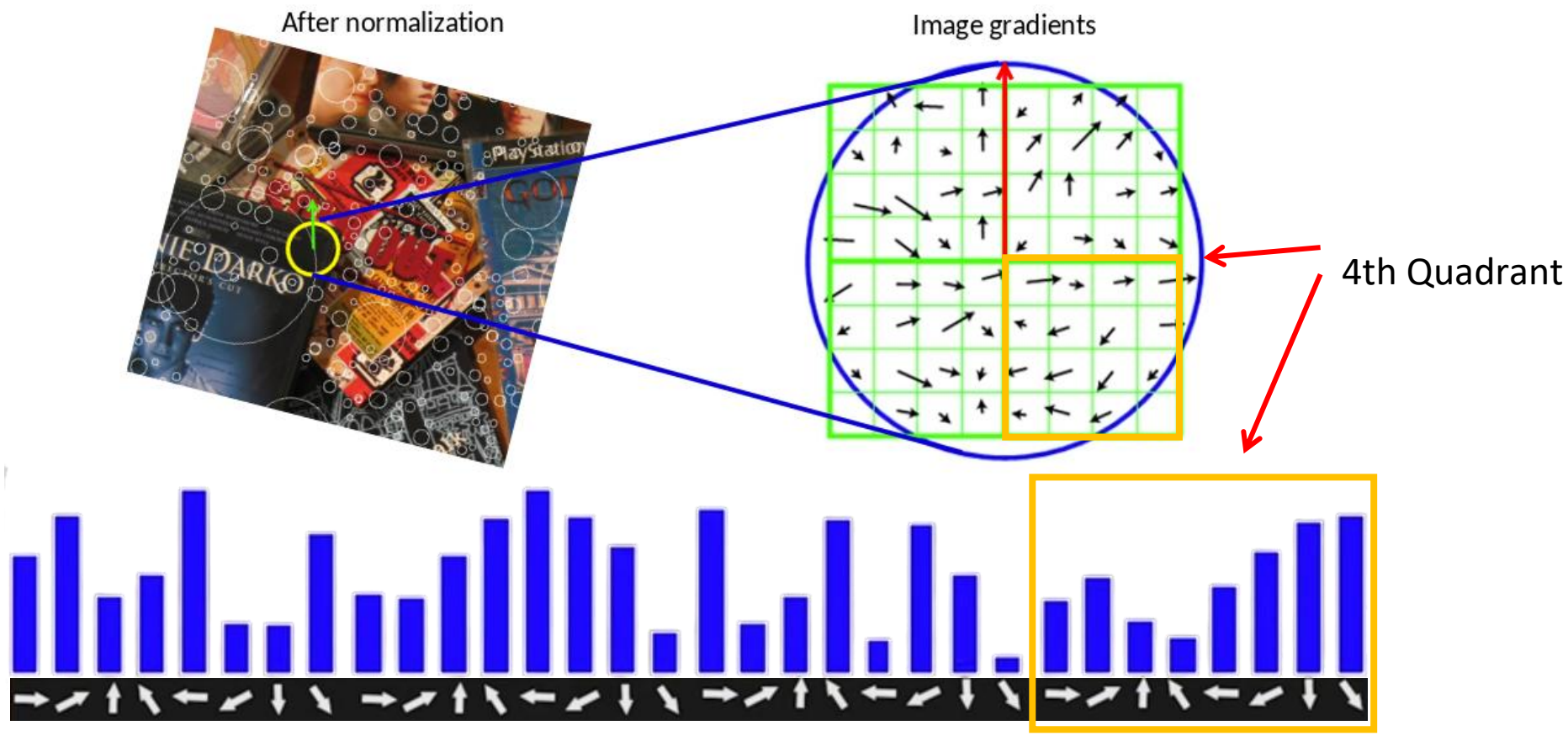
The grid is divided into quadrants, and for each quadrant we compute a gradient orientation histogram — counting how many pixels point in each direction.

SIFT Descriptor



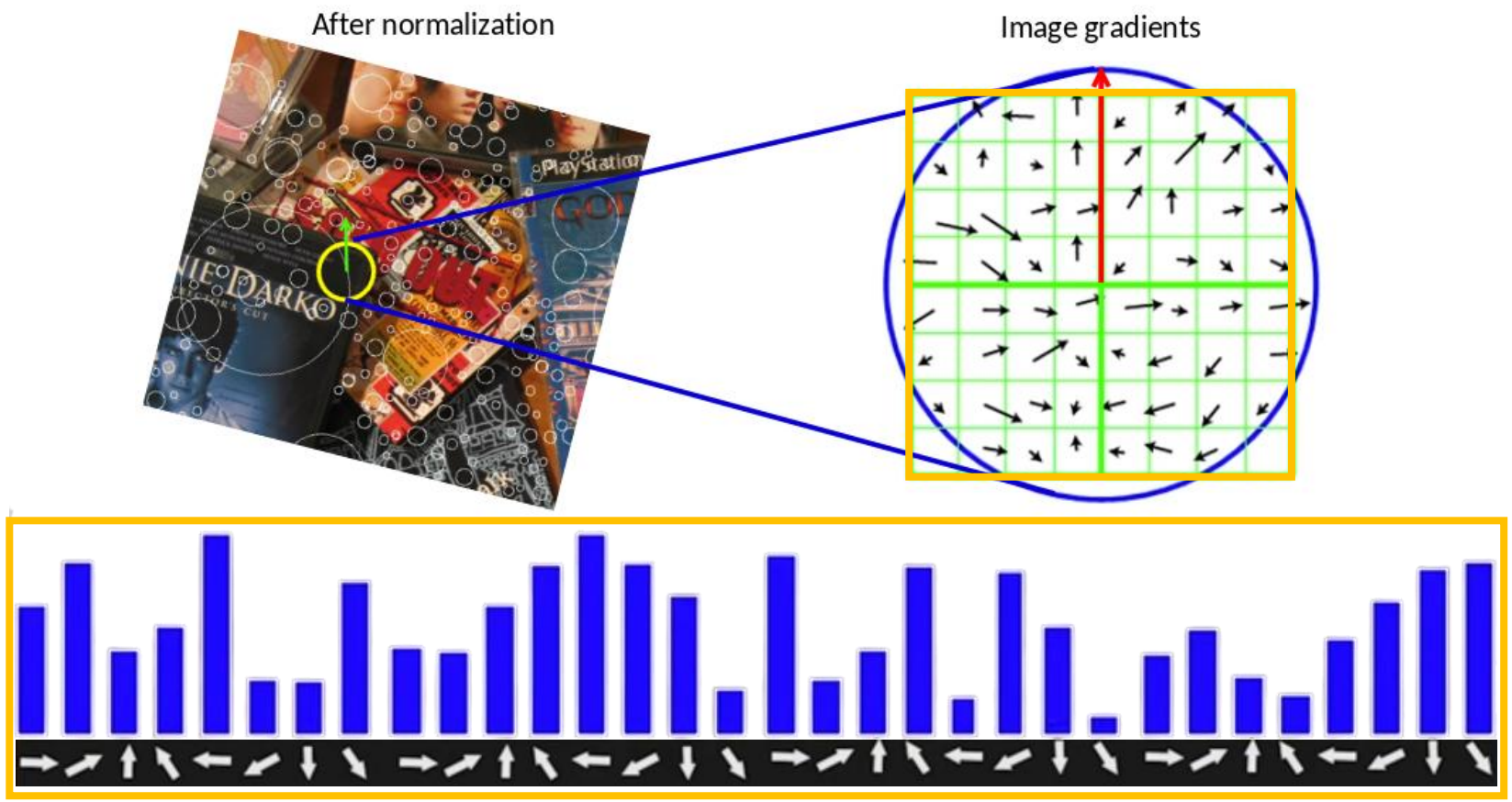
The grid is divided into quadrants, and for each quadrant we compute a gradient orientation histogram — counting how many pixels point in each direction.

SIFT Descriptor



The grid is divided into quadrants, and for each quadrant we compute a gradient orientation histogram — counting how many pixels point in each direction.

SIFT Descriptor



The orientation histograms from all quadrants are concatenated into a single descriptor — **invariant to rotation, scale, and robust to brightness changes** — ready for matching.

Comparing SIFT Descriptors

Essentially comparing two arrays of data.

Let $H_1(k)$ and $H_2(k)$ be two arrays of data of length N.

L2 Distance:

$$\boxed{d(H_1, H_2) = \sqrt{\sum_k \left(H_1(k) - H_2(k)\right)^2}}$$

Smaller the distance metric, better the match.

Perfect match when $d(H_1, H_2) = 0$

Comparing SIFT Descriptors

Essentially comparing two arrays of data.

Let $H_1(k)$ and $H_2(k)$ be two arrays of data of length N.

Normalized Correlation:

$$\boxed{d(H_1, H_2) = \frac{\sum_k [(H_1(k) - \bar{H}_1)(H_2(k) - \bar{H}_2)]}{\sqrt{\sum_k (H_1(k) - \bar{H}_1)^2}\sqrt{\sum_k (H_2(k) - \bar{H}_2)^2}}}$$

where: $\bar{H}_i = \frac{1}{N}\sum_{k=1}^{N} H_i(k)$

Larger the distance metric, better the match.

Perfect match when $d(H_1, H_2) = 1$

Comparing SIFT Descriptors

Essentially comparing two arrays of data.

Let $H_1(k)$ and $H_2(k)$ be two arrays of data of length N.

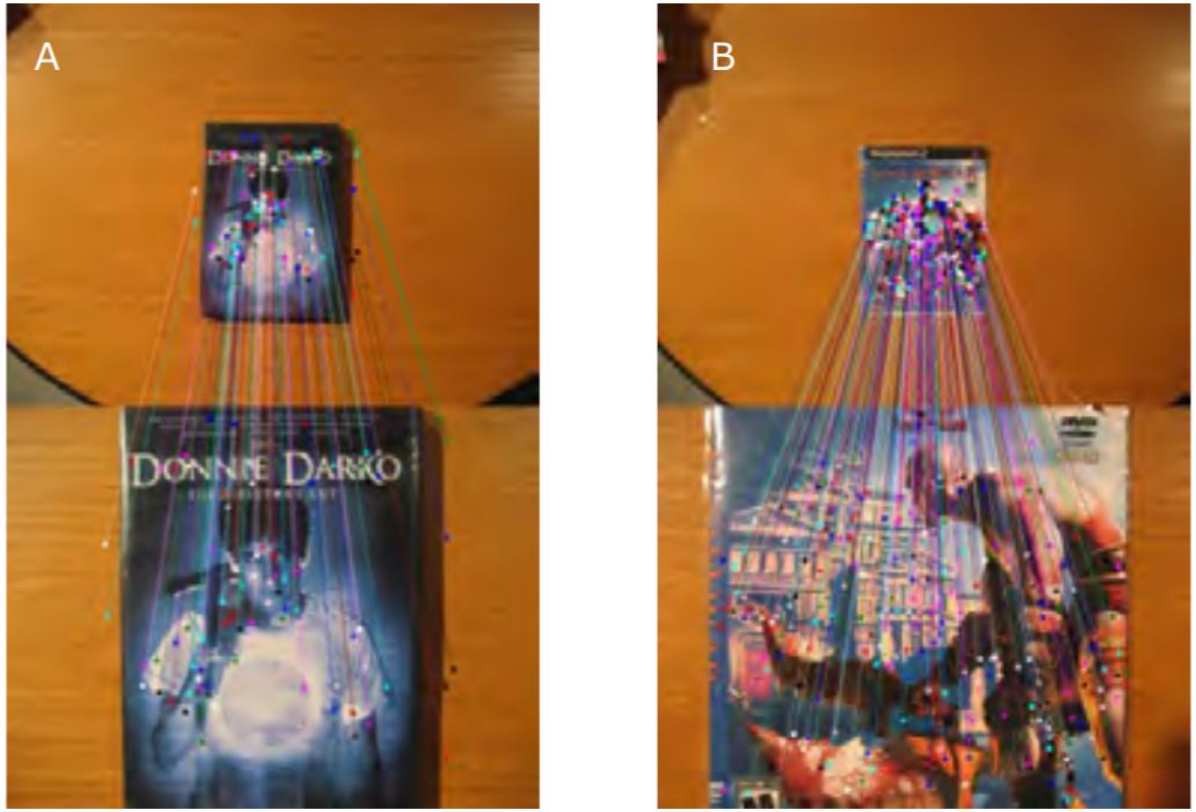
Intersection:

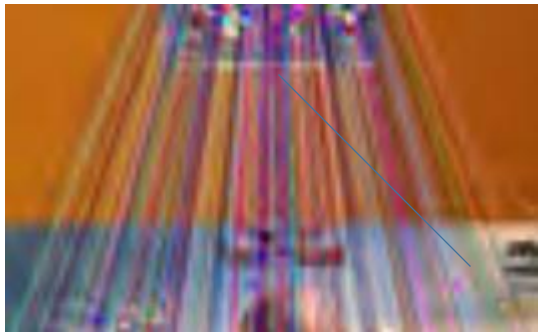
$$\boxed{d(H_1, H_2) = \sum_k \min\big(H_1(k), H_2(k)\big)}$$

Larger the distance metric, better the match.

Note: The intersection metric compares two descriptors by summing the bin-wise minimum of their histograms.

SIFT Results: Scale Invariance

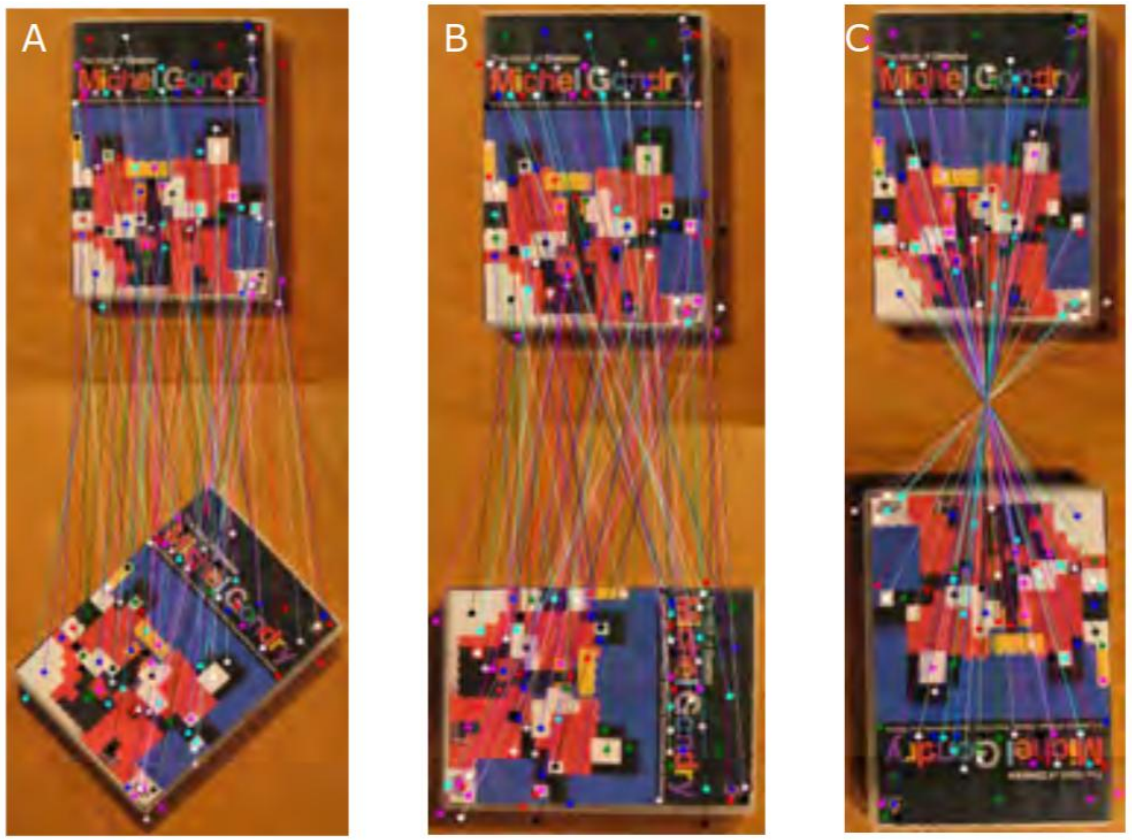




correct matches

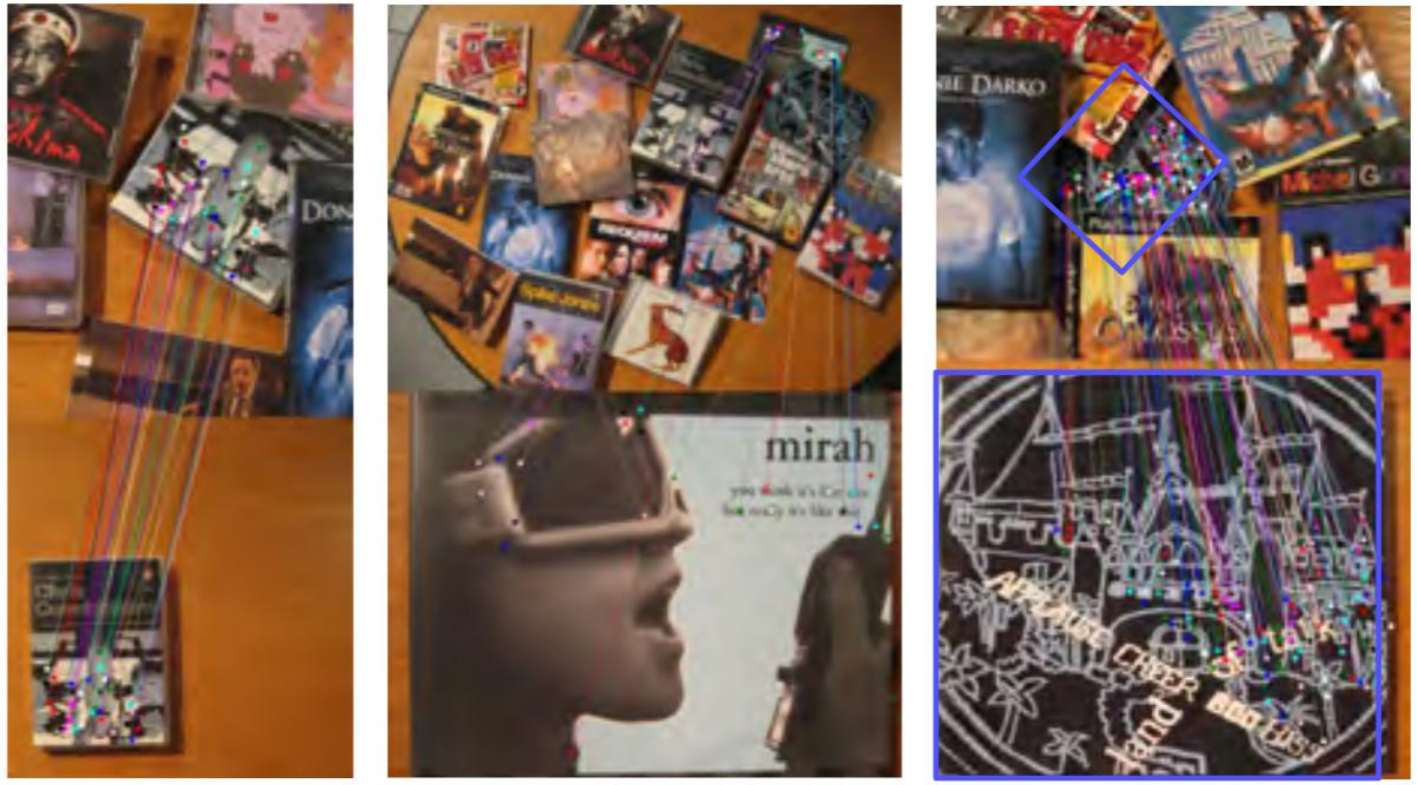
SIFT successfully matches features across images of the same object at different **scales** — each line represents a correct **match** between two corresponding **interest points**, demonstrating SIFT's **scale invariance**.

SIFT Results: Rotation Invariance



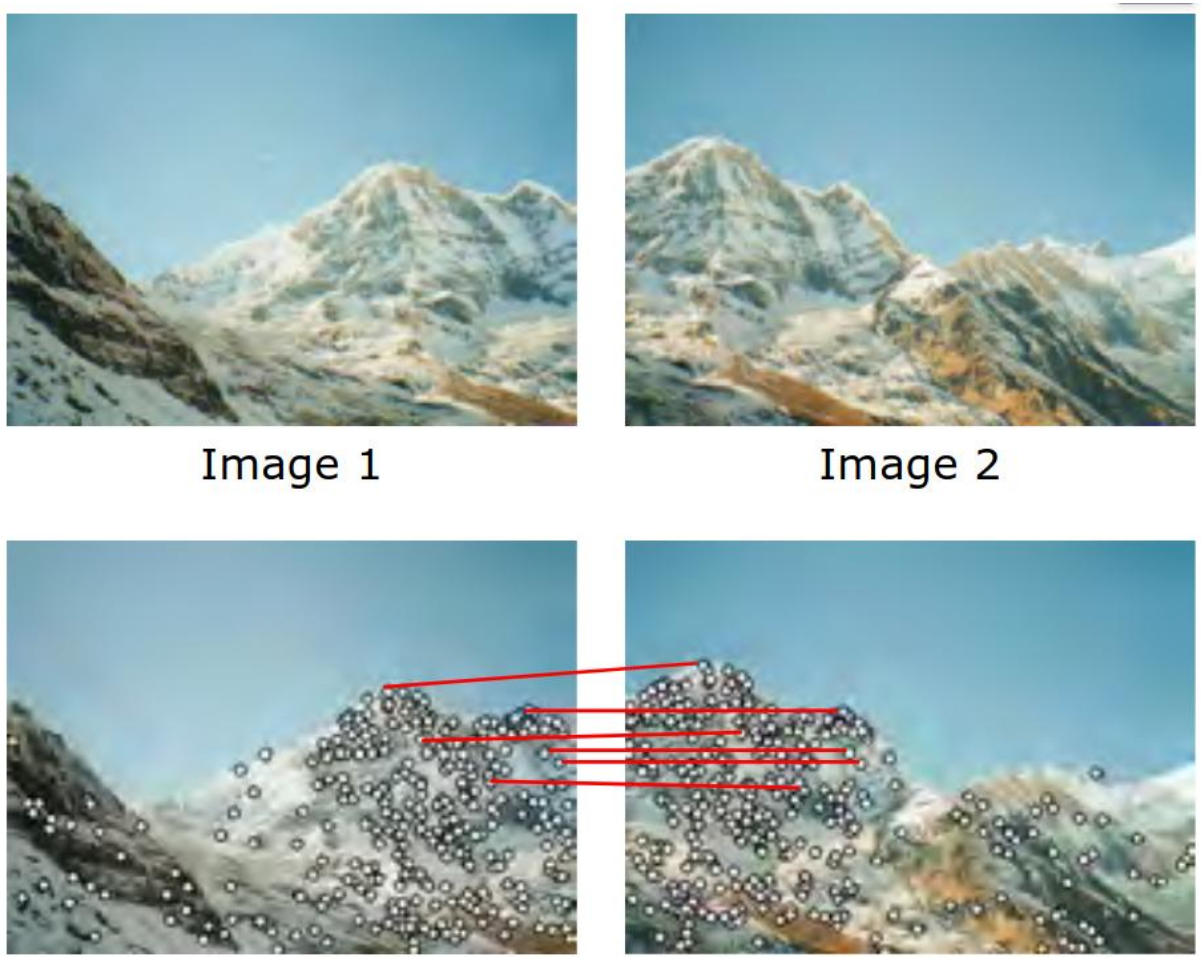
SIFT achieves reliable **matching** across both **scale** and **rotation** changes — even at 90 degrees or fully upside down, a large number of correct **matches** are found, demonstrating SIFT's **rotation invariance**.

SIFT Results: Robustness to Clutter



Even under **partial occlusion**, SIFT successfully matches the **visible features** of the object without **mismatches** with other objects — demonstrating SIFT's robustness to **clutter** and **occlusion**.

Panorama Stitching using SIFT



Autostitch

Image stitching uses SIFT by detecting and **matching features** across two overlapping images, then applying a **geometric transformation** to **warp** and **align** them — producing a larger seamless **panorama**.

Brown, M., & Lowe, D. G. (2007). Automatic panoramic image stitching using invariant features. *International journal of computer vision*, *74*(1), 59-73.

Panorama Stitching using SIFT

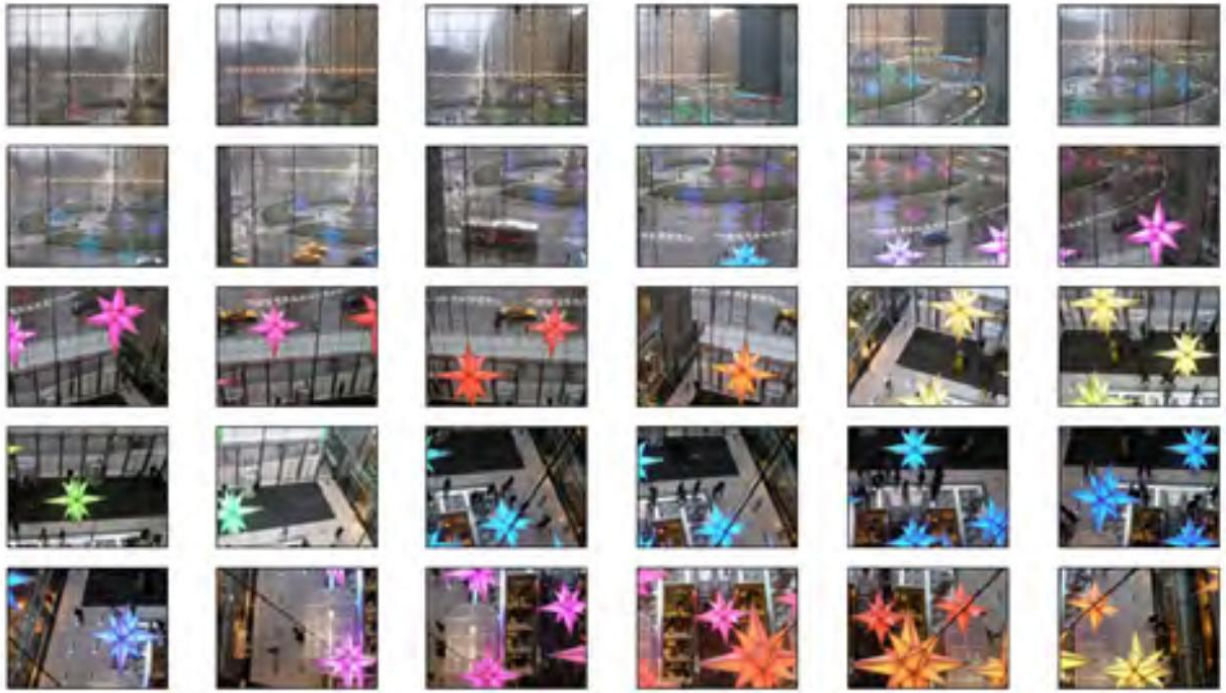
Warp and combine images to create a larger image

Autostitch

Image stitching uses SIFT by detecting and **matching features** across two overlapping images, then applying a **geometric transformation** to **warp** and **align** them — producing a larger seamless **panorama**.

Brown, M., & Lowe, D. G. (2007). Automatic panoramic image stitching using invariant features. *International journal of computer vision*, *74*(1), 59-73.

Auto Collage using SIFT



Captured Photos

Auto Collage

SIFT can also be used to construct **collages** — by detecting and **matching features** across a large collection of images and bringing them together into a single image with a much wider **field of view**.

Nomura, Y., Zhang, L., & Nayar, S. K. (2007, June). Scene collages and flexible camera arrays. In Proceedings of the 18th Eurographics conference on Rendering Techniques (pp. 127-138).

Auto Collage using SIFT

Auto Collage

SIFT can also be used to construct **collages** — by detecting and **matching features** across a large collection of images and bringing them together into a single image with a much wider **field of view**.

Nomura, Y., Zhang, L., & Nayar, S. K. (2007, June). Scene collages and flexible camera arrays. In Proceedings of the 18th Eurographics conference on Rendering Techniques (pp. 127-138).